



Rapport de Stage de première année de master Systèmes Embarqués - Systèmes Intégrés

AUTEUR

GUILLEMOD Quentin

MAÎTRE DE STAGE

EUSTACHE Yvan



2026-06-22

Remerciements

Je remercie Yvan Eustache, mon maître de stage au Lab-STICC, pour m'avoir confié ce projet avec une réelle autonomie et pour sa disponibilité tout au long du stage. Je remercie également Teddy Robert pour sa participation aux tests de portée LoRa réalisés au port de Lorient.

Table des matières

1	Introduction	4
2	Présentation du laboratoire d'accueil	5
2.1	Présentation générale du Lab-STICC	5
2.2	Domaines de recherche	5
2.3	Organisation et environnement de travail	5
2.4	Lien avec la formation	6
3	Contexte et objectifs du projet	6
4	Technologies, outils et protocoles utilisés	8
4.1	Langages de programmation	8
4.2	Capteurs et protocoles utilisés	8
4.3	Outils supplémentaires utilisés	11
5	Analyse du projet initial	13
5.1	Méthode d'analyse	15
5.2	État fonctionnel	15
5.3	Qualité technique du code et de l'architecture	15
5.4	Aspects organisationnels	16
5.5	Décision de refonte	16
6	Conception de la solution	17
6.1	Démarche de conception	18
6.2	Architecture globale du système	18
6.2.1	Réseau de noeuds embarqués	18
6.2.2	PC central	18
6.3	Architecture logicielle côté serveur	19
6.4	Gestion de la configuration	19
7	Réalisation et développement	19
7.1	Introduction	19
7.2	Mise en place du système de build et d'installation automatique	20
7.3	Extraction du code HTML/CSS/JavaScript	20
7.4	Correction des bibliothèques et mise en local	20
7.5	Centralisation des paramètres	20
7.6	Refonte du code Python côté serveur	20
7.7	Refonte du code C++ embarqué	21
7.8	Refonte de la visualisation et des protocoles d'échange client/serveur	21

7.9	Ajout d'un système de logs	21
7.10	Passage sous Docker	21
7.11	Chiffrement des communications	22
7.12	Intégration du capteur accéléromètre (MinIMU-9 v6)	22
7.13	Intégration du capteur sonore (INMP441)	22
7.14	Ajout des tests de simulation de données	22
7.15	Système de gestion d'événements et configuration d'avant-match	23
7.16	Passage sous Meshcore	25
7.17	Tests de portée et de stabilité LoRa	25
7.18	Conception des PCBs et des modèles 3D	25
7.19	Intégration des balises Ultra Wideband (DWM3001CDK)	25
7.20	Tests de simulation de tracking	25
8	Validation et évaluation	26
8.1	Test de distance de portée	26
8.2	Test de mise à l'échelle	27
8.3	Validation du système de communication LoRa	28
9	Bilan du projet et apports du stage	28
9.1	Bilan du projet	28
9.2	Perspectives d'amélioration	29
9.3	Apports techniques	29
9.4	Apports organisationnels	29
10	Conclusion	30
11	Sources	31
12	Annexes	33
.1	Structure et organisation du code du projet	33
.2	Capture d'écran de l'interface web du serveur	37
.3	Schéma des PCBs	40
.4	Design des PCBs	42
.5	Design des modèles 3D	44
.6	Document de tests	46

1 Introduction

L'analyse des comportements des supporters en conditions réelles est un problème ouvert. Les données disponibles se limitent généralement aux statistiques de vente de billets ou aux sondages post-match, loin de capturer ce qui se passe réellement dans les tribunes. C'est dans ce contexte que le Lab-STICC et le Football Club de Lorient ont initié un projet visant à collecter des données physiologiques et comportementales de supporters pendant un match, en temps réel et sans dépendance à une infrastructure réseau externe.

L'idée de ce projet est donc de suivre les réactions des supporters pendant le match et de les associer aux événements survenant lors du match. Ce projet combine l'Internet des Objets (IoT), les systèmes embarqués mais également la science des données. L'objectif de ce projet est de réaliser une vidéo promotionnelle valorisant le laboratoire de recherche de Lorient, le club de foot ainsi que la ville, en s'appuyant sur des données réelles collectées auprès des supporters.

Le stage que j'ai réalisé au Lab-STICC d'une durée de 3 mois, s'inscrit dans le stage de fin d'année du Master 1 Systèmes Embarqués - Systèmes Intégrés. Ce projet de monitoring de données des supporters a été commencé par des étudiants de Master 2 Systèmes Embarqués - Systèmes Intégrés, lors de leur projet de premier semestre. Les missions confiées lors de ce stage ne se limitaient pas à étendre le dispositif existant. Face aux limitations structurelles du projet initial, il a été nécessaire de repartir de zéro. Et il a aussi été nécessaire de reconcevoir l'architecture logicielle dans son intégralité, d'intégrer de nouveaux capteurs et de préparer le système à un déploiement réel en conditions extérieures.

Ce rapport présente l'ensemble des travaux réalisés lors de mon stage. Il décrit dans un premier temps le laboratoire d'accueil, le contexte du stage et les technologies et outils utilisés tout au long du développement de la solution. Mais détaille aussi les analyses et conception de l'infrastructure, la réalisation des tests mis en place pour la validation de la solution. Puis il termine par un bilan sur ce que ce stage m'a apporté.

2 Présentation du laboratoire d'accueil

2.1 Présentation générale du Lab-STICC

Le Laboratoire des Sciences et Techniques de l'Information, de la Communication et de la Connaissance (Lab-STICC) est une unité mixte de recherche (UMR 6285) associée au Centre National de la Recherche Scientifique (CNRS) et à plusieurs établissements d'enseignement supérieur bretons. Le laboratoire mène des activités de recherche dans les domaines des sciences et technologies de l'information et de la communication.

Le Lab-STICC est réparti sur plusieurs sites en Bretagne, notamment à Brest, Lorient, Quimper et Vannes. Cette organisation favorise les collaborations interdisciplinaires entre chercheurs, enseignants-chercheurs, doctorants et partenaires industriels.

Le laboratoire développe des travaux de recherche allant de l'acquisition des données à leur exploitation pour l'aide à la décision. Ses activités s'inscrivent dans une approche globale visant à transformer des données issues de capteurs ou de systèmes d'information en connaissances exploitables.

2.2 Domaines de recherche

Les recherches menées au sein du Lab-STICC couvrent de nombreux domaines scientifiques liés au numérique et aux systèmes complexes. Parmi les principales thématiques étudiées figurent :

- Les systèmes embarqués et les architectures matérielles
- Les réseaux de communication et les systèmes connectés
- Le traitement du signal et des données
- L'intelligence artificielle et l'analyse de données
- Les systèmes interactifs et les interfaces homme-machine
- La cybersécurité
- L'aide à la décision et la modélisation de systèmes complexes

Ces compétences permettent au laboratoire d'intervenir dans différents secteurs d'application tels que le domaine maritime, les transports, la santé, la défense, l'industrie ou encore les technologies numériques innovantes.

2.3 Organisation et environnement de travail

Le Lab-STICC rassemble une communauté scientifique composée de chercheurs, enseignants-chercheurs, doctorants, ingénieurs et personnels techniques. Cette diversité favorise les échanges de

compétences et le développement de projets multidisciplinaires.

Au cours de mon stage, j'ai évolué dans un environnement de recherche appliquée où les problématiques scientifiques sont étroitement liées à des besoins concrets. Cette immersion m'a permis de découvrir le fonctionnement d'un laboratoire de recherche ainsi que les méthodes de travail associées à la conduite de projets technologiques.

2.4 Lien avec la formation

Le stage réalisé au sein du Lab-STICC s'inscrit directement dans les thématiques abordées au cours du Master Systèmes Embarqués et Systèmes Intégrés. Les travaux réalisés mobilisent des compétences liées à l'acquisition de données, au développement logiciel, à l'intégration de systèmes communicants et à l'analyse d'informations issues de dispositifs connectés.

Cette expérience constitue ainsi une mise en application concrète des connaissances acquises durant mes formations scolaires tout en permettant le développement de compétences complémentaires telles que l'autonomie, la gestion de projet, la résolution de problématiques techniques complexes ou encore la mise en place de protocoles de tests.

3 Contexte et objectifs du projet

Ce stage s'inscrit dans le cadre d'une collaboration entre le Lab-STICC de Lorient et le Football Club de Lorient (FCL). L'objectif global du projet est de développer une solution IoT performante, économique et à gestion intégralement locale des données, destinée à analyser les données physiologiques et comportementales de supporters pendant des matchs de football. Les données collectées ont vocation à alimenter une vidéo promotionnelle valorisant le club, la ville de Lorient et le laboratoire de recherche. La contrainte de gestion locale est centrale, le dispositif doit fonctionner intégralement hors connexion internet, depuis la collecte des données jusqu'à leur visualisation. Ce qui exclut tout recours à des services cloud ou à des bibliothèques chargées à la volée. De plus il doit respecter les normes européennes pour les transmissions radio, ainsi qu'avoir une faible consommation énergétique, et doit garantir une certaine sécurité des données des supporters qui est en majorité assurée par la gestion locale de ces données. Et certains nœuds sont destinés à être portés par les supporters, ce qui implique que le dispositif doit être le plus petit possible et ne pas gêner les supporters pendant le match.



FIGURE 1 – Photo du stade du Moustoir avec une capacité de 18 100 personnes

Le projet a été initialement développé par un groupe d'étudiants de Master 2 Systèmes Embarqués - Systèmes Intégrés, encadrés par deux ingénieurs du Lab-STICC. À l'issue de leurs travaux, le dispositif permettait de visualiser en temps réel la fréquence cardiaque de deux supporters via des capteurs Polar H10 connectés à des nœuds Meshtastic, les données étant acheminées par LoRa vers un PC central chargé de leur traitement. Leur base était fonctionnelle mais elle présentait des limitations sérieuses rendant toute évolution du projet risquée et coûteuse.

Avant le début officiel du stage, j'ai eu l'opportunité de travailler sur ce projet pendant un mois dans le cadre d'un projet encadré. Cette période m'a permis de prendre connaissance du code existant, d'analyser ses faiblesses et de commencer à définir les orientations techniques de la refonte.

Dans ce contexte, mon stage de trois mois ainsi que le mois de projet avaient pour mission de reprendre entièrement le projet. Les objectifs étaient les suivants :

- Reconcevoir l'architecture logicielle from scratch, en garantissant la modularité, la maintenabilité, le fonctionnement hors ligne intégral, et une installation automatisée fiable sur Linux.
- Étendre le dispositif à de nouveaux types de capteurs : capteurs sonores, accéléromètres, et balises de localisation Ultra Wideband.
- Faire passer à l'échelle le réseau de nœuds, le dispositif devant être capable de gérer simultanément plus de dix supporters équipés, chacun transmettant plusieurs flux de données en temps réel via LoRa.
- Valider la solution en conditions extérieures, par des tests en environnement ouvert hors laboratoire, afin de préparer un déploiement réel lors d'un match du FCL prévu après la fin du stage.

L'ensemble de ces travaux mobilise des compétences transversales couvrant le développement embarqué en C++ sur cartes ESP32, le développement serveur en Python, la programmation web en HTML/CSS et JavaScript, la gestion de bases de données hétérogènes (InfluxDB pour les données temps réel, PostgreSQL pour les données événementielles), ainsi que l'intégration de protocoles de communication variés (BLE, I2S, I2C, LoRa, MQTT, WebSocket).

4 Technologies, outils et protocoles utilisés

4.1 Langages de programmation

Les langages utilisés dans ce projet couvrent l'ensemble de la chaîne, du firmware embarqué jusqu'à l'interface utilisateur.

Le C++ est utilisé pour le développement des firmwares des cartes Xiao Seeed ESP32. Ce choix se justifie pour plusieurs raisons. Premièrement il s'agit du langage déjà retenu dans le projet initial. De plus il offre des performances élevées et une faible consommation mémoire, qui sont deux contraintes critiques sur des microcontrôleurs dû à leurs ressources limitées. Et enfin il bénéficie d'un écosystème de bibliothèques Arduino/ESP32 particulièrement riche pour l'interaction avec les capteurs et les protocoles de communication.

Le Python est le langage principal côté serveur. Il est utilisé pour l'application Flask, les bridges Meshtastic/Meshcore vers MQTT, le bridge MQTT vers InfluxDB, ainsi que pour le parsing des données issues des scripts de Qorvo. Son utilisation se justifie par sa présence dans le projet initial, par sa simplicité de mise en œuvre pour un serveur web comparé à des alternatives comme Java ou PHP, et par la richesse de son écosystème de bibliothèques.

Le HTML/CSS et JavaScript sont utilisés pour le développement de l'interface web cliente. Le choix de ces technologies sans framework frontend se justifie par une maîtrise solide du trio de base et par l'absence de connaissance approfondie des frameworks. Chart.js est intégré côté client pour la visualisation graphique des données en temps réel directement dans le navigateur.

Le Bash est utilisé pour les scripts d'automatisation. Comme le chiffrement du fichier de configuration, lecture de logs et flashage des cartes.

4.2 Capteurs et protocoles utilisés

Plusieurs types de capteurs ont été intégrés au dispositif, chacun utilisant un protocole de communication adapté à ses contraintes. Les capteurs ont été imposés dans le cadre du projet, leur sélection n'a donc pas fait l'objet d'une justification ou d'une comparaison avec d'autres capteurs aux

caractéristiques similaires.

Les capteurs de fréquence cardiaque sont les Polar H10 de Polar Electro. Ils communiquent avec les cartes ESP32 via Bluetooth Low Energy (BLE), protocole adapté aux échanges sans fil à faible consommation énergétique sur courte portée, et nativement supporté par les capteurs Polar.



FIGURE 2 – Capteur Polar H10

Les capteurs sonores sont les INMP441 de InvenSense. Ils utilisent le protocole I2S pour transmettre les données audio numériques à l'ESP32. I2S est le protocole standard pour la transmission audio numérique sur microcontrôleur, offrant une communication synchrone et fiable.



FIGURE 3 – Capteur INMP441

Les accéléromètres sont les MinIMU-9 v6 de Pololu. Ils communiquent avec l'ESP32 via le bus I2C, protocole série synchrone permettant de connecter plusieurs périphériques sur un même bus avec

seulement deux fils, ce qui simplifie le câblage sur un noeud embarqué.

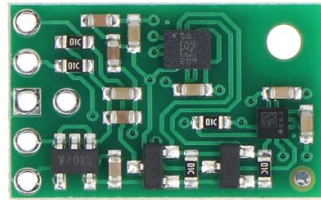


FIGURE 4 – Capteur MinIMU-9 v6

Les balises Ultra Wideband sont les DWM3001CDK de Qorvo. Contrairement aux autres capteurs, elles opèrent en dehors du réseau LoRa. Elles permettent, via trilatération, de calculer la position d'une balise Tag par rapport à des balises Ancres fixes. Les données sont récupérées en parcourant directement les sorties des scripts fournis par Qorvo.



FIGURE 5 – Carte DWM3001CDK

L'ensemble des nœuds ESP32 communique vers le PC central via LoRa, géré par Meshtastic ou Meshcore selon la configuration. LoRa est particulièrement adapté à ce projet car il permet des communications longue portée à faible consommation énergétique, sans infrastructure réseau préexistante. Ce qui est cohérent avec la contrainte de fonctionnement hors ligne. Meshcore n'est cependant pas encore pleinement opérationnel dû à un problème de changement de mode du port USB lors du flashage sur ESP32 provoquant l'interruption du chargement complet du firmware. Une fois les données reçues par le PC central, elles sont distribuées via un broker MQTT avant d'être stockées en base de données.



FIGURE 6 – Carte Xiao seed studio ESP32 S3 Plus

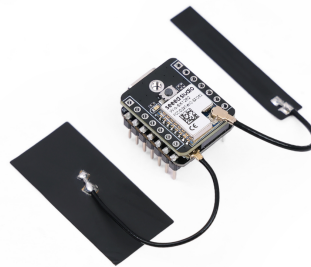


FIGURE 7 – Carte Xioa seed studio ESP32 S3 avec module LoRa Wio SX1262

4.3 Outils supplémentaires utilisés

Au-delà des langages et des capteurs, le projet s'appuie sur un ensemble d'outils couvrant le flashage des cartes, l'isolation des processus, la gestion et le stockage des données, la visualisation, la sécurité et l'automatisation. Ces outils ont été sélectionnés pour répondre aux contraintes du projet :

- Le fonctionnement hors ligne
- La reproductibilité de l'environnement
- La facilité de déploiement sur site
- La maintenabilité à long terme

PlatformIO est utilisé pour le flashage des firmwares et des configurations sur les cartes ESP32. Il offre une gestion des dépendances et un environnement de compilation reproductible, ce qui est particulièrement adapté à un projet multi-cartes nécessitant des configurations différentes selon les

nœuds.

Docker est utilisé pour isoler les différents processus du serveur (broker MQTT, bases de données, application Flask, bridges) dans des conteneurs indépendants. Cela garantit la reproductibilité de l'environnement d'exécution, facilite les tests et simplifie le déploiement sur une nouvelle machine, ce qui est essentiel pour un dispositif destiné à être installé sur site.

MQTT, via un broker Mosquitto, est utilisé comme couche de communication interne entre les différents composants serveur. Son modèle publish/subscribe permet de découpler proprement les producteurs de données (bridges LoRa) des consommateurs (bases de données, interface web). Ce protocole était déjà présent dans le projet initial et s'avère bien adapté aux architectures IoT grâce à sa légèreté.

InfluxDB est utilisé pour le stockage des données capteurs. Son modèle orienté séries temporelles est particulièrement adapté à des volumes importants de données horodatées, en temps réel et non modifiables, générées en continu par les nœuds du réseau.

PostgreSQL est utilisé pour le stockage des données événementielles, structurées et potentiellement modifiables. Le choix d'une base de données relationnelle se justifie par la nature de ces données, peu volumineuses mais fortement structurées. PostgreSQL a été retenu spécifiquement pour expérimenter un système différent de MySQL et SQLite déjà maîtrisés, et en comprendre les avantages et inconvénients dans un contexte réel.

Grafana est utilisé en parallèle de l'interface web pour la visualisation des données stockées dans InfluxDB. Il offre des tableaux de bord configurables sans développement spécifique, utiles notamment en phase de développement et de validation pour inspecter rapidement les données brutes. Son maintien dans l'architecture du projet se justifie par sa présence dans la version initiale, ainsi que par sa pertinence pour l'analyse et la supervision des données collectées.

Flask avec SocketIO constitue le serveur web de l'application. Flask est un micro-framework Python léger, suffisant pour les besoins du projet et cohérent avec le choix de Python côté serveur. SocketIO permet une communication bidirectionnelle en temps réel avec le navigateur via WebSockets, nécessaire pour l'affichage live des données capteurs et des événements liés au match.

SOPS (Secrets OPerationS) est utilisé pour le chiffrement par clé GPG du fichier `.env`, qui centralise l'ensemble de la configuration du projet. Cela permet de versionner ce fichier dans Git sans exposer les secrets qu'il contient, tout en maintenant un workflow de développement simple. L'ajout de cet outil se justifie particulièrement par la gestion de mes projets personnels, qui utilise ce mode de chiffrement de fichier d'environnement.

Git est utilisé pour la gestion de versions du projet, permettant le suivi des modifications, la gestion des branches et la traçabilité de l'ensemble des développements réalisés.

Make est utilisé comme outil d'automatisation global du projet. Il centralise en un point d'entrée unique l'ensemble des commandes fréquemment utilisées. Comme le lancement des scripts, l'installation des dépendances, le nettoyage, la suppression, la gestion de Docker ou encore la lecture des logs. Sa simplicité et son omniprésence dans les environnements Linux en font un choix naturel pour ce type de projet.

5 Analyse du projet initial

Avant d'entamer toute phase de développement, une analyse approfondie du projet existant a été nécessaire. Cette analyse a porté sur trois axes :

- L'état fonctionnel du dispositif
- La qualité technique du code et de l'architecture
- Les aspects organisationnels tels que la documentation et la gestion du projet

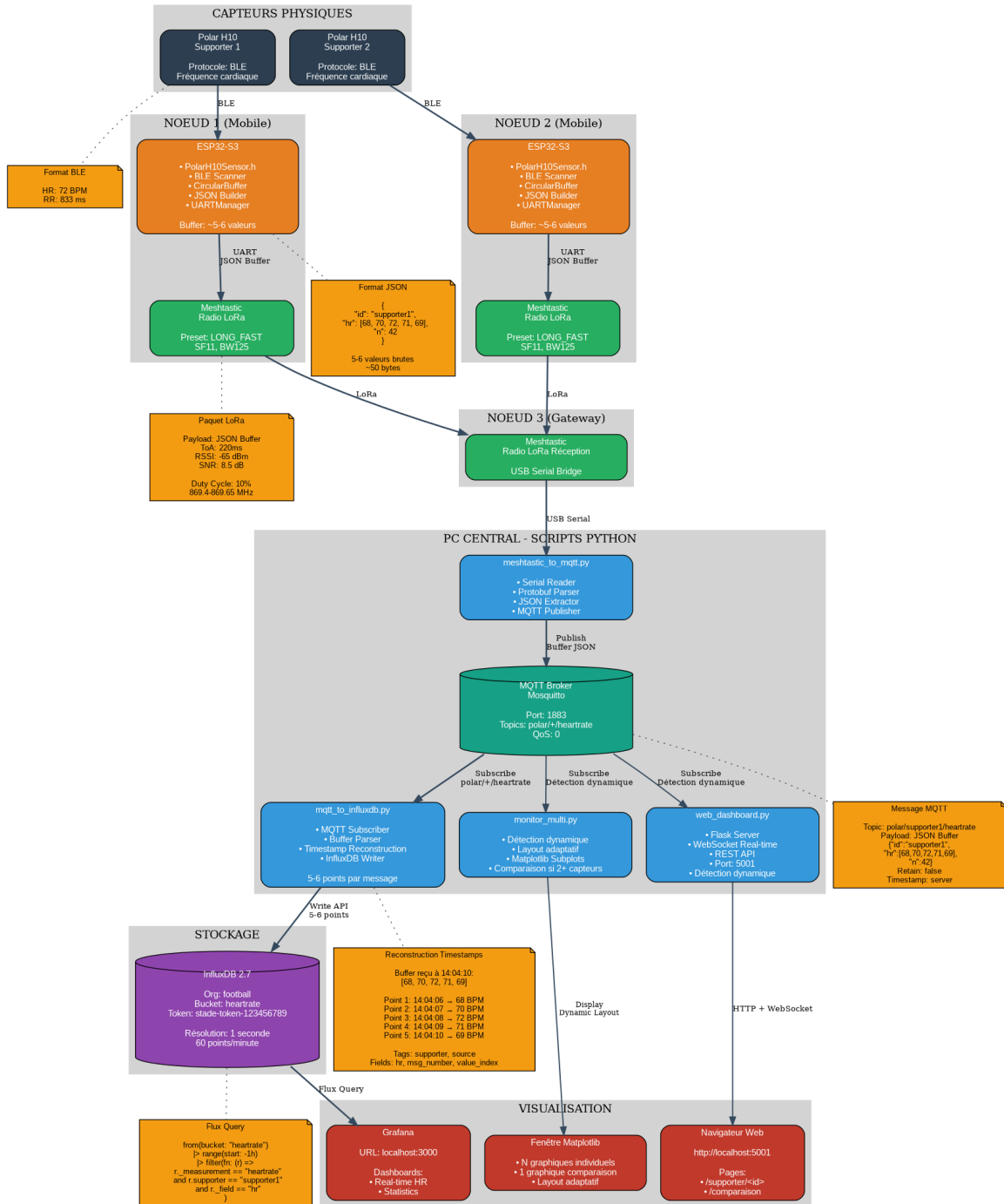


FIGURE 8 – Architecture du projet initial

5.1 Méthode d'analyse

L'analyse s'est déroulée en quatre étapes successives. En premier lieu, la documentation fournie a été lue et la structure générale du projet examinée, afin d'obtenir une vision globale de l'architecture avant de plonger dans le code. En second lieu, l'installation du dispositif a été tentée en suivant les instructions disponibles, ce qui a immédiatement mis en évidence plusieurs dysfonctionnements. En troisième lieu, des tests fonctionnels ont été réalisés sur les parties opérationnelles pour évaluer ce qui fonctionnait réellement. Enfin, une lecture transversale du code a permis d'identifier les problèmes structurels sous-jacents.

Cette analyse a été menée sans aucun échange possible avec les anciens étudiants M2, rendant la compréhension de certains choix techniques uniquement déductible à partir du code et de la documentation disponibles.

5.2 État fonctionnel

À la reprise du projet, le dispositif permettait de visualiser en temps réel la fréquence cardiaque de deux supporters via des capteurs Polar H10, chacun connecté à un nœud Meshtastic. Une gateway recevait les données LoRa et les transmettait à une application Flask chargée de leur traitement et de leur affichage. Cette base était fonctionnelle dans des conditions contrôlées, mais limitée à un seul type de capteur et à deux nœuds simultanés, ce qui était insuffisant au regard des objectifs du projet. Le script d'installation automatique, censé fonctionner sur macOS et Linux, n'était pas entièrement opérationnel sous Linux, ce qui rendait le déploiement du dispositif sur une nouvelle machine non fiable sans intervention manuelle. Par ailleurs, les bibliothèques JavaScript utilisées par l'interface web étaient chargées à l'exécution depuis des CDN internet. Cette dépendance est incompatible avec la contrainte fondamentale du projet qui impose un fonctionnement intégral hors connexion. Le dispositif ne pouvait donc pas être utilisé tel quel dans les conditions réelles d'un match.

5.3 Qualité technique du code et de l'architecture

L'analyse du code a révélé plusieurs problèmes structurels rendant toute évolution risquée et coûteuse.

La modularité affichée du code était en réalité fondée sur de la duplication. Les mêmes blocs de logique étaient recopiés à plusieurs endroits plutôt que factorisés, ce qui implique qu'une correction ou une modification doit être répercutée manuellement en plusieurs points, avec un risque élevé d'incohérence. Les configurations n'étaient pas centralisées. Les paramètres étaient dispersés dans le code, rendant toute modification de l'environnement (adresses, ports, identifiants) fastidieuse et source d'erreurs. Le code HTML de l'interface était directement injecté dans des variables Python sous forme de chaînes de caractères. Cette approche rend le code illisible, impossible à déboguer et non maintenable dès lors que l'interface gagne en complexité. La séparation des responsabilités

côté serveur était insuffisante. La logique métier, la gestion des routes, la communication avec les bases de données et le rendu de l'interface étaient mélangés, sans architecture claire. De même que pour l'interface BLE intégrée au capteur de fréquence cardiaque, ce qui n'est pas pratique en cas de réutilisation du Bluetooth pour un autre capteur ou une autre fonctionnalité.

Enfin, une partie significative du code présentait les caractéristiques d'un code généré par intelligence artificielle sans relecture critique : structures redondantes, gestion des erreurs absente ou superficielle, logique parfois incohérente avec le contexte du projet et présence de caractères typique tels que des icônes dans les chaînes de caractères. L'absence de gestion d'erreurs robuste est particulièrement problématique pour un dispositif destiné à fonctionner en conditions extérieures, où les défaillances matérielles et les pertes de communication sont inévitables. Aucun système de logs structuré n'était en place, ce qui rendait le diagnostic des problèmes en cours d'exécution très difficile, voire impossible sans modifier le code.

5.4 Aspects organisationnels

Le projet n'était pas versionné avec Git. En l'absence d'historique des modifications, il est impossible de tracer les évolutions du code, de revenir sur une version antérieure stable ou de comprendre les intentions derrière certains choix techniques. La documentation fournie était peu exploitable, elle entraînait dans un niveau de détail relevant normalement des commentaires dans le code lui-même, sans offrir de vision claire du fonctionnement global du système, de son architecture ou des choix techniques effectués. Cette absence de vue d'ensemble m'a conforté dans ma décision de refonte complète. Les tests réalisés étaient insuffisants et non formalisés, ce qui ne permettait pas d'évaluer objectivement la robustesse du dispositif ni d'identifier les régressions lors de modifications.

5.5 Décision de refonte

Face à l'ensemble de ces constats, la décision a été prise de repartir de zéro plutôt que de tenter de corriger l'existant. Le coût d'une correction aurait été supérieur à celui d'une refonte propre, et les problèmes structurels identifiés, en particulier la duplication de code, l'absence de séparation des responsabilités et l'injection de code HTML dans Python, n'étaient pas corrigeables sans réécriture complète. Les éléments conservés du projet initial se limitent aux choix technologiques fondamentaux : le C++ pour l'embarqué, Python côté serveur, et les bibliothèques Flask et SocketIO.

6 Conception de la solution

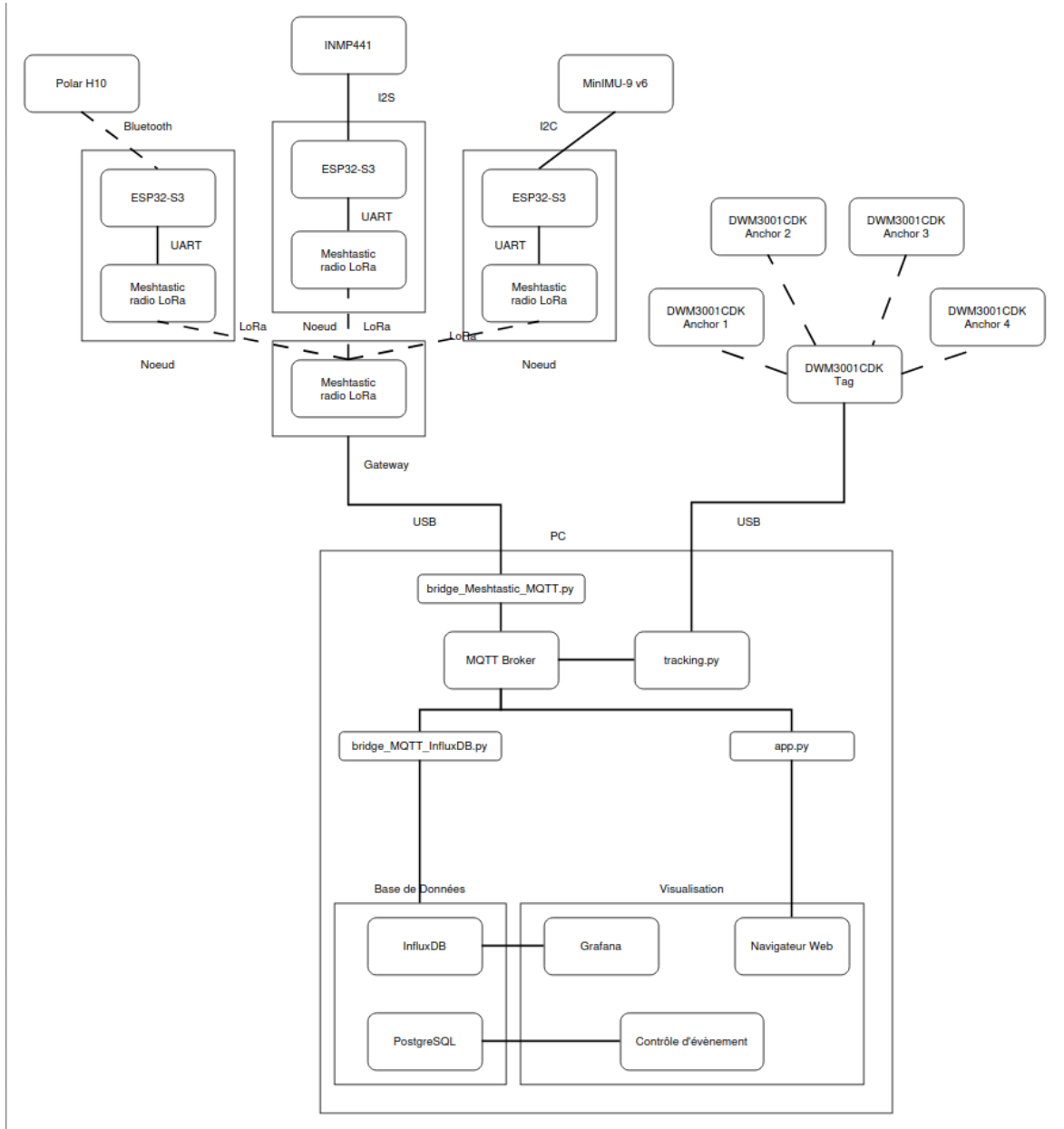


FIGURE 9 – Architecture du projet

6.1 Démarche de conception

La conception de la solution a suivi une démarche structurée en trois étapes. L'analyse du projet initial a d'abord permis d'identifier précisément les besoins fonctionnels et les contraintes techniques à respecter. Ces besoins ont ensuite été formalisés sous forme d'exigences claires avant d'être traduits en choix d'architecture. Enfin, l'architecture a été schématisée avant tout développement, afin de disposer d'une vision globale cohérente du système et d'éviter les incohérences structurelles qui avaient affecté le projet initial.

6.2 Architecture globale du système

Le système est organisé en deux grandes parties distinctes : le réseau de noeuds embarqués chargés de la collecte des données, et le PC central chargé de leur réception, traitement, stockage et affichage.

6.2.1 Réseau de noeuds embarqués

Chaque noeud est composé d'une carte Xiao Seeed ESP32 associée à un unique type de capteur. Ce choix de spécialisation des nœuds garantit la simplicité des firmwares, facilite le débogage et permet une configuration indépendante de chaque nœud sans risque d'interférence entre les logiques de capteurs. Un nœud dédié à la fréquence cardiaque, un autre au son, un autre à l'accélérométrie.

Le flux de données au sein d'un nœud suit le chemin suivant : le capteur transmet ses données à l'ESP32 via son protocole natif (BLE, I2S ou I2C selon le capteur). L'ESP32 les transmet ensuite via UART à une seconde carte ESP32 exécutant le firmware Meshtastic, qui se charge de les acheminer par LoRa jusqu'à la gateway centrale. Ce découplage entre la carte de collecte et la carte de communication LoRa permet de faire évoluer l'un ou l'autre indépendamment.

Les balises Ultra Wideband constituent un cas particulier. Elles opèrent en dehors du réseau LoRa et communiquent directement avec le PC central, leurs données étant parsées depuis les sorties des scripts Qorvo.

6.2.2 PC central

La gateway reçoit les trames LoRa et les publie sur le broker MQTT. À partir de ce point, les données sont consommées par plusieurs composants indépendants selon leur nature :

- Les données capteurs (séries temporelles) sont écrites dans InfluxDB via un bridge MQTT dédié.
- Les données événementielles (incidents, annotations de match) sont écrites dans PostgreSQL.
- L'interface web reçoit les données en temps réel via WebSocket grâce à Flask et SocketIO.

Ce modèle publish/subscribe via MQTT découple proprement les producteurs de données des consommateurs, rendant l'ajout d'un nouveau consommateur (une nouvelle base, un nouvel affichage) transparent pour le reste du système.

6.3 Architecture logicielle côté serveur

Le code serveur est organisé selon une séparation stricte des responsabilités. Chaque fichier Python a un rôle unique et clairement défini : gestion des routes Flask, logique métier, communication avec InfluxDB, communication avec PostgreSQL, gestion des WebSockets. Cette structure évite les dépendances circulaires et permet de modifier ou tester un composant sans impacter les autres.

L'ensemble des services serveur est conteneurisé via Docker, avec un conteneur par service : broker MQTT (Mosquitto), InfluxDB, PostgreSQL, application Flask, et chaque bridge de données. Cette isolation garantit la reproductibilité de l'environnement d'exécution et simplifie le déploiement sur site, indépendamment de la machine hôte.

6.4 Gestion de la configuration

L'ensemble de la configuration du projet est centralisé dans un unique fichier `.env`, lu par tous les composants au démarrage. Ce fichier est chiffré par clé GPG via SOPS, ce qui permet de le versionner dans Git sans exposer les secrets qu'il contient (identifiants, ports, clés). Le déchiffrement est intégré aux commandes Make, rendant l'opération transparente pour l'utilisateur. Et pour les configurations complexes telles que l'initialisation de la base de données et les configurations via format JSON sont centralisées dans un dossier nommé `Resources`. Permettant de ne pas intégrer ces fichiers ou variables de configurations directement dans le code.

Ce choix de centralisation répond directement à l'un des problèmes identifiés dans le projet initial, où les paramètres de configuration étaient dispersés dans le code, rendant toute modification d'environnement risquée.

7 Réalisation et développement

7.1 Introduction

Le développement a suivi un ordre chronologique dicté par les priorités de stabilisation avant extension. La première phase a consisté à assainir et restructurer la base existante avant d'y ajouter de nouvelles fonctionnalités. Ce choix a permis de travailler sur des fondations saines à chaque nouvelle étape, évitant de propager les problèmes structurels du projet initial dans les nouveaux développements.

7.2 Mise en place du système de build et d'installation automatique

La première action a été la mise en place de Make comme système de build et point d'entrée unique du projet. Avant tout développement, disposer d'une automatisation fiable des opérations courantes était indispensable : lancement et arrêt des services, installation des dépendances, flashage des cartes, lecture des logs, nettoyage de l'environnement. Cette décision prise dès le départ a considérablement simplifié toutes les étapes suivantes, chaque nouvelle opération étant immédiatement intégrée sous forme de cible Make.

7.3 Extraction du code HTML/CSS/JavaScript

La seconde étape a consisté à extraire le code HTML, CSS et JavaScript des chaînes de caractères Python dans lesquelles il était directement injecté. Chaque template a été déplacé dans des fichiers dédiés, servis par Flask via son mécanisme de rendu de templates Jinja2. Cette séparation a immédiatement rendu le code frontend lisible, débogable avec les outils standards du navigateur, et maintenable indépendamment du code serveur.

7.4 Correction des bibliothèques et mise en local

Les dépendances JavaScript chargées depuis des CDN internet ont été téléchargées et intégrées localement au projet. Cette étape était bloquante pour respecter la contrainte de fonctionnement hors ligne. Sans elle, l'interface web était inutilisable dans les conditions réelles d'un match.

7.5 Centralisation des paramètres

L'ensemble des paramètres de configuration dispersés dans le code ont été centralisé dans un unique fichier `.env`. Ce fichier est chiffré par clé GPG via SOPS, ce qui permet de le versionner dans Git sans exposer les secrets qu'il contient. Tous les composants du projet lisent leur configuration depuis ce fichier au démarrage, garantissant une cohérence totale de l'environnement et simplifiant les modifications de configuration. Pour les configurations plus complexes au format JSON, elles ont été écrites dans un dossier spécifique pour faciliter l'accès à ces configurations.

7.6 Refonte du code Python côté serveur

Le code serveur a été intégralement réécrit avec une séparation stricte des responsabilités. Routes Flask, logique métier, accès InfluxDB, accès PostgreSQL et gestion des WebSockets sont répartis dans des fichiers distincts aux rôles clairement définis. Les bridges Meshtastic vers MQTT et MQTT vers InfluxDB ont également été réécrits en tant que processus Python indépendants, chacun avec un périmètre fonctionnel unique.

7.7 Refonte du code C++ embarqué

Les firmwares C++ des cartes ESP32 ont été intégralement réécrits. La principale difficulté rencontrée lors de cette étape concernait la gestion BLE pour le capteur Polar H10. La bibliothèque que j'utilisais dans le projet s'est avérée obsolète, provoquant des instabilités de communication. La migration vers une bibliothèque plus récente a résolu ces instabilités, mais a introduit un problème résiduel, le temps de détection d'une déconnexion du capteur qui est d'environ 40 secondes. Ce délai, lié aux mécanismes internes du protocole BLE, n'a pas pu être réduit davantage et a été accepté comme contrainte connue du système.

7.8 Refonte de la visualisation et des protocoles d'échange client/serveur

Le protocole d'échange de données entre le serveur et l'interface web a été revu et formalisé. Les données capteurs sont transmises en temps réel via WebSocket grâce à SocketIO, dès leur réception par le serveur depuis MQTT. Chart.js est utilisé côté client pour leur visualisation graphique. Ce flux a été conçu pour être extensible. L'ajout d'un nouveau type de capteur ne nécessite que l'ajout d'un nouveau topic MQTT et d'un nouveau composant graphique, sans modifier l'architecture existante.

7.9 Ajout d'un système de logs

Un système de logs structuré a été mis en place à deux niveaux. Côté serveur, chaque composant Python produit des logs horodatés et classés par niveau de sévérité, accessibles. Côté embarqué, un système de logs sur le système de fichiers flash de l'ESP32 a été implémenté via LittleFS, permettant de conserver une trace des événements survenus sur la carte sans connexion série. Les appels API natifs de LittleFS n'étant pas reconnus à l'exception de l'initialisation, la solution retenue a été d'utiliser les appels système POSIX standards (`open`, `write`, `close`), qui fonctionnent correctement au-dessus de LittleFS sur ESP32. Ces logs sont lisibles grâce un dump des logs sur UART lors du démarrage de la carte et au script de récupération de logs.

7.10 Passage sous Docker

L'ensemble des services serveur a été conteneurisé :

- broker MQTT (Mosquitto)
- Grafana
- InfluxDB
- PostgreSQL
- application Flask
- bridges de données
- les tests

Chacun dispose de son propre conteneur, isolé des autres. Cette isolation garantit la reproductibilité de l'environnement d'exécution et simplifie le déploiement sur site. Le démarrage complet de l'infrastructure se fait via une unique commande Make.

7.11 Chiffrement des communications

Les transmissions LoRa sont diffusées en clair sur un canal radio public. Sans chiffrement, tout récepteur compatible peut intercepter les paquets transitant entre les nœuds et la gateway, ce qui est incompatible avec la contrainte de confidentialité des données des supporters. Pour couvrir ce risque, un chiffrement AES-256 en mode CTR a été configuré sur l'ensemble du réseau Meshtastic. Une clé partagée est distribuée à tous les nœuds et à la gateway au moment du flashage. Cette clé est stockée dans le fichier `.env` centralisé et chiffré par GPG via SOPS, ce qui évite de l'exposer dans le dépôt Git tout en maintenant un workflow de déploiement simple. Cette mesure couvre l'interception passive des données en transit. Elle ne couvre pas l'usurpation d'identité d'un nœud ni les attaques sur le PC central, dont la sécurité repose sur l'isolation réseau locale et la gestion des accès à la machine hôte.

7.12 Intégration du capteur accéléromètre (MinIMU-9 v6)

L'accéléromètre MinIMU-9 v6 de Pololu a été le premier nouveau capteur intégré. La communication I2C avec l'ESP32 est bien supportée. Un firmware dédié a été développé et validé sans difficulté majeure, bénéficiant de la solidité de l'architecture embarquée mise en place lors de la refonte.

7.13 Intégration du capteur sonore (INMP441)

Le microphone INMP441 utilise le protocole I2S pour transmettre les données audio numériques à l'ESP32. La principale contrainte de ce capteur réside dans le volume des données produites, significativement supérieur à celui des autres capteurs. Une attention particulière a été portée à la gestion des buffers mémoire sur l'ESP32 et à la définition du format de transmission des données sonores via LoRa.

7.14 Ajout des tests de simulation de données

Des scripts de simulation de données ont été développés afin de pouvoir tester les communications et les interactions avec le serveur sans déployer le système physique complet. Ces scripts génèrent des données aléatoires mais réalistes pour chaque type de capteur et les injectent directement dans le broker MQTT, simulant le comportement de nœuds réels. Cela a permis de valider le comportement du serveur, des bridges et de l'interface web de manière indépendante du matériel.

7.15 Système de gestion d'événements et configuration d'avant-match

Le système d'événements a constitué la partie la plus complexe du développement côté serveur. Il permet de créer, modifier et supprimer des événements liés au match en temps réel ou a posteriori, tout en maintenant la cohérence des données visuelles mais également par rapport aux données relatives à ce même événement. Plusieurs contraintes ont été traitées :

- La cohérence des données lors de modifications ou suppressions.
- La récupération de l'état courant complet lors d'une reconnexion ou d'une réouverture de page, afin qu'un client rejoignant en cours de match dispose immédiatement de l'historique complet.
- La configuration personnalisée des types d'événements, chaque match pouvant avoir ses propres paramètres.
- La phase de préparation d'avant-match, synchronisée avec le système d'événements, permettant de configurer le contexte (équipes, stade, paramètres) avant le début de la collecte.

Ces données sont stockées dans PostgreSQL, dont le modèle relationnel est adapté à leur structure fortement typée et potentiellement modifiable.



FIGURE 10 – Capture d'écran de la page web de configuration du contrôle des événements



FIGURE 11 – Capture d'écran de la page web de préparation d'avant match



FIGURE 12 – Capture d'écran de la page web du contrôle des événements du match

7.16 Passage sous Meshcore

Une tentative de passage sous Meshcore a été réalisée en remplacement de Meshtastic pour la gestion du réseau LoRa. Meshcore n'est cependant pas encore pleinement opérationnel dû à un problème de changement de mode du port USB lors du flashage sur ESP32, qui provoque l'interruption du chargement complet du firmware. Le projet supporte les deux configurations, mais Meshtastic reste la configuration principale opérationnelle à ce stade.

7.17 Tests de portée et de stabilité LoRa

Des tests en environnement extérieur ont été réalisés pour évaluer la portée maximale du réseau LoRa et la stabilité des connexions à distance variable. Ces tests ont permis de valider le comportement du réseau dans des conditions proches de celles d'un stade et d'identifier les limites du système en termes de distance entre les nœuds et la gateway.

7.18 Conception des PCBs et des modèles 3D

Afin de disposer de nœuds complets et autonomes, les PCBs des nœuds ont été conçus sous EasyEDA. Ces PCBs intègrent la carte ESP32, le capteur associé et les composants nécessaires à l'alimentation et à la connexion des éléments. Les boîtiers des nœuds ont quant à eux été modélisés sous OnShape, afin de protéger le matériel en conditions extérieures et de faciliter le port du dispositif par les supporters. Le design des PCBs ainsi que les modèles 3D ont été conçus pour être les plus petits possibles afin de ne pas gêner le supporter et d'être également pratiques pour recharger la batterie ou récupérer des informations de debug via les ports séries des ESP32. La plus grosse contrainte de ce modèle a été l'intégration de la batterie, car c'est l'élément le plus volumineux du système. Il n'est donc pas possible d'avoir un boîtier plus petit que les dimensions de la batterie. Voir les schémas .3, PCBs .4 et modèles 3D .5 en annexes.

7.19 Intégration des balises Ultra Wideband (DWM3001CDK)

Les balises Ultra Wideband DWM3001CDK de Qorvo ont été intégrées en dernier. Contrairement aux autres capteurs, elles opèrent en dehors du réseau LoRa et communiquent directement avec le PC central. Les données de localisation sont récupérées en parsemant les sorties des scripts fournis par Qorvo, permettant de calculer par trilatération la position des balises Tag par rapport aux balises Ancres fixes.

7.20 Tests de simulation de tracking

Des tests de simulation du système de tracking Ultra Wideband ont été réalisés pour valider l'affichage des trajectoires sur l'interface web. Ces tests ont permis de vérifier la cohérence des données de localisation et la fluidité de leur visualisation en temps réel avant tout déploiement terrain. La gestion du calcul de la position et de la visualisation du tracking est modulaire par rapport aux

nombres de balises Ancre, à la forme de la zone et aussi à la position de chacune des balises. Mais elle reste dépendant d'un plan 2D.

8 Validation et évaluation

8.1 Test de distance de portée

Afin d'évaluer la portée de la communication LoRa, plusieurs protocoles de tests composés d'une carte émettrice et d'une carte réceptrice ont été mis en place. L'émetteur envoyait périodiquement (10s) des paquets de taille fixe contenant des données similaires à celles réellement transmises par le système de monitoring. La distance entre les deux cartes a ensuite été augmentée progressivement afin d'observer l'évolution de la qualité de transmission. Voir les annexes avec les images GPS des points de tests (parking de Science 1 .6, port de Lorient .6)

Les premiers essais ont été réalisés sur le parking du centre de recherche et de la Faculté des Sciences, avec des distances comprises entre 100 et 150 mètres. Deux configurations LoRa prédéfinies de Meshtastic ont été testées. LONG_FAST, optimisée pour des transmissions rapides sur de longues distances, et SHORT_FAST, destinée aux transmissions rapides sur des distances plus courtes. Les résultats obtenus avec ces deux configurations ont été similaires. Les paquets ont été transmis correctement, avec un signal de bonne qualité et aucune perte notable. Les essais se sont poursuivis jusqu'à l'intérieur du bâtiment Sciences 1, où les premières pertes de paquets ont commencé à apparaître.

Afin d'évaluer les performances sur de plus grandes distances, une seconde campagne de tests a été réalisée au port de Lorient. Les essais ont débuté à une distance de 250 mètres avec une ligne de vue directe entre l'émetteur et le récepteur. Dans ces conditions, aucun paquet n'a été perdu et la qualité du signal est restée satisfaisante. La ligne de vue a été conservée sur la totalité du trajet.

Un dernier test a été effectué à une distance de 800 mètres en ligne de vue directe. Les résultats ont montré une transmission fiable, sans perte de paquets, malgré une légère diminution de la puissance du signal reçu.

Cette phase de test a également permis d'évaluer les performances en absence de ligne de vue. En effet, le trajet permettant d'atteindre les 800 mètres ne garantissait pas une visibilité permanente entre les deux équipements. Il a été constaté qu'à partir d'environ 400 mètres hors ligne de vue, le signal devenait plus faible, bien qu'aucune perte de paquets n'ait été observée. En revanche, à partir de 600 mètres, le taux de réception diminuait significativement, avec seulement un paquet reçu sur deux en moyenne.

À la suite de ces résultats, des essais complémentaires ont été menés en augmentant la puissance d'émission LoRa afin de déterminer si cette modification pouvait améliorer les performances hors ligne de vue. Les résultats obtenus se sont toutefois révélés moins satisfaisants. Des pertes de paquets sont apparues même à 800 mètres en ligne de vue, alors qu'aucune perte n'avait été observée précédemment. De plus, hors ligne de vue, la réception s'est dégradée dès 400 mètres avec un taux de réception proche de 60%, tandis qu'à partir de 600 mètres, d'importantes interruptions de communication ont été constatées, correspondant à des pertes successives de 3 à 6 paquets.

Ces essais de portée ont été réalisés avec la participation de EUSTACHE Yvan pour les essais menés sur le parking du centre de recherche et de la Faculté des Sciences, ainsi que de ROBERT Teddy pour les essais effectués au port de Lorient. Voir les graphiques des résultats des tests de distances en annexe .6

8.2 Test de mise à l'échelle

Afin d'évaluer le comportement des communications LoRa en présence de plusieurs émetteurs fonctionnant simultanément, une première série de tests a été réalisée avec cinq nœuds émetteurs répartis dans un périmètre restreint d'environ 100 mètres.

Les résultats obtenus se sont révélés très éloignés des attentes. La réception était principalement limitée à des groupes de paquets provenant d'un seul nœud à la fois, avec un changement occasionnel du nœud reçu. De nombreuses pertes de paquets ont également été observées. Par ailleurs, durant toute la durée de l'expérimentation, aucun paquet provenant de l'un des cinq émetteurs n'a été reçu.

L'analyse des logs a mis en évidence une saturation du réseau pendant le test. L'hypothèse retenue pour expliquer ce phénomène est liée au mécanisme de retransmission utilisé par Meshtastic. En effet, lorsqu'un nœud transmet un message, celui-ci peut être relayé par d'autres nœuds du réseau si le destinataire final n'est pas directement atteint. Chaque message est alors retransmis à plusieurs reprises, ce qui multiplie le trafic généré et peut rapidement conduire à une saturation du canal de communication.

Afin de vérifier cette hypothèse, un second test a été réalisé avec le même nombre d'émetteurs, mais en désactivant la fonction de retransmission des messages. Les résultats obtenus ont été nettement plus satisfaisants, l'ensemble des messages émis par les cinq nœuds a été reçu correctement, sans perte de paquets observée.

Ces résultats tendent à confirmer que le mécanisme de relais est à l'origine de la saturation constatée lors du premier essai. Toutefois, il reste nécessaire d'étudier le comportement du réseau lors d'une montée en charge avec un nombre plus important d'émetteurs. En effet, même sans retransmission, l'augmentation du trafic généré pourrait à terme entraîner une nouvelle saturation des communications LoRa.

8.3 Validation du système de communication LoRa

Ces tests construisent une bonne base fonctionnelle et fiable des communications LoRa, mais ne sont évidemment pas suffisant pour garantir le bon fonctionnement. Ces tests nous permettent seulement de valider la distance de portée, car un stade de football n'aura pas besoin d'une couverture réseau de plus de 200m. Cependant un test au stade pour vérifier le comportement des ondes radio serait plus fiable car la structure des tribunes, la disposition et le mouvement du public impactent la propagation des ondes radio et donc sa réception. De plus le test de mise à l'échelle effectué avec seulement cinq capteurs est un bon début pour valider le fonctionnement de plusieurs émetteurs simultanés mais n'est clairement pas suffisant si nous voulons monter la charge à une centaine de capteurs.

La plus grosse contrainte de ces transmissions LoRa réside dans le respect du duty cycle, pour avoir une infrastructure légale. Le duty cycle implique une limitation du temps de transmission radio sur une heure, en France nous sommes limités à 10%. Nous devons donc porter une attention particulière sur ce point car plus nous avons de capteurs plus le temps de transmission augmente. Cependant le duty cycle dépend des channels de communication, il est donc possible d'utiliser plusieurs canaux de communication pour faire descendre le temps de transmission de chacun des canaux à 10% et ainsi respecter la norme.

9 Bilan du projet et apports du stage

9.1 Bilan du projet

À l'issue du stage, la solution développée couvre l'ensemble des objectifs initialement fixés : La refonte complète de l'architecture logicielle, l'intégration de quatre types de capteurs, le passage à l'échelle du réseau de nœuds, la conteneurisation de l'infrastructure serveur, le système d'événements et de configuration d'avant-match, la conception des PCBs et des modèles 3D des nœuds, et les premiers tests en environnement extérieur.

Le point manquant est l'absence de validation en conditions réelles au stade. La fin de la saison du FCL avant la clôture du stage n'a pas permis d'organiser un déploiement lors d'un match. Cette limite est contraignante car une partie des choix d'architecture, notamment en ce qui concerne la portée LoRa en environnement dense et le comportement du système face à un grand nombre de supporters équipés simultanément, repose sur des validations en environnement ouvert et non sur des conditions réelles de stade. Ce test reste la prochaine étape critique du projet.

9.2 Perspectives d'amélioration

Si le projet était à recommencer, la priorité serait de définir des protocoles de tests dès le début du développement et de chercher à tester le dispositif en conditions réelles le plus tôt possible. Baser les choix d'architecture sur des hypothèses théoriques ou des tests en environnement contrôlé présente un risque. Certaines contraintes sont propres à un environnement de stade (densité humaine, interférences radio, distances réelles entre nœuds et gateway), qui pourrait invalider des décisions qui semblaient correctes sur le papier. Une confrontation terrain précoce aurait permis d'ajuster l'architecture avant que celle-ci ne soit figée.

9.3 Apports techniques

Ce stage a été l'occasion de découvrir et de mettre en pratique plusieurs technologies jusqu'alors inconnues. Docker et la conteneurisation ont été appréhendés dans un contexte concret, avec la gestion d'une infrastructure multi-services et les problématiques d'isolation, de communication inter-conteneurs et de déploiement reproductible. PostgreSQL a été découvert et comparé aux systèmes relationnels déjà maîtrisés (MySQL, SQLite), permettant d'en comprendre les spécificités et les avantages dans un contexte de données événementielles structurées. LittleFS sur ESP32 a introduit la gestion d'un système de fichiers sur microcontrôleur, avec les contraintes spécifiques liées aux appels système dans cet environnement. La conception de PCBs sous EasyEDA et de modèles 3D sous OnShape a constitué une première expérience complète de conception matérielle, de la schématisation électronique jusqu'au boîtier physique. Enfin, Meshtastic et Meshcore ont permis d'appréhender les réseaux maillés LoRa et leurs contraintes d'intégration dans un système embarqué.

Au-delà des nouvelles technologies, ce stage a consolidé des compétences déjà acquises sur des sujets plus avancés. Comme la gestion de la communication temps réel via WebSockets, l'intégration de protocoles embarqués variés (BLE, I2S, I2C, LoRa), l'architecture logicielle avec séparation des responsabilités, et la sécurisation de la configuration via chiffrement GPG.

9.4 Apports organisationnels

Ce stage a représenté une première expérience significative de conduite autonome d'un projet technique sur une longue durée. La gestion des priorités, le découpage du travail en étapes cohérentes et la capacité à prendre des décisions techniques sans pouvoir s'appuyer sur les développeurs initiaux ont constitué des apprentissages concrets. La mise en place dès le départ d'outils de gestion de projet (Git, Make, Docker, documentation) a démontré son utilité tout au long du développement, chaque composant restant accessible, modifiable et déployable indépendamment.

10 Conclusion

Ce stage a abouti à une refonte complète et fonctionnelle d'un dispositif IoT de monitoring de supporters, couvrant l'ensemble de la chaîne. Du firmware embarqué sur cartes ESP32 jusqu'à l'interface web de visualisation en temps réel, en passant par l'infrastructure serveur conteneurisée, les bridges de données, le système de gestion d'événements et la conception des PCBs et boîtiers 3D. Les quatre types de capteurs sont intégrés, le réseau LoRa est opérationnel avec Meshtastic, et le système a été validé en environnement extérieur. Les contraintes centrales du projet : un fonctionnement intégral hors connexion internet, une consommation énergétique basse et la garantie de la confidentialité des données des supporters, sont respectées à tous les niveaux de la solution.

La limite principale de ce stage reste l'absence de validation en conditions réelles au stade du FCL. La fin de saison du club a rendu ce test impossible avant la clôture du stage. C'est le point le plus important à traiter en priorité. C'est en conditions réelles que certaines hypothèses d'architecture, comme la portée LoRa en environnement dense, le comportement avec plus de dix nœuds simultanés ou encore la résistance aux interférences radio qui seront véritablement confrontées à la réalité. Les tests en environnement ouvert réalisés constituent une étape intermédiaire nécessaire, mais pas suffisante.

Sur le plan personnel, ce stage a représenté une expérience significative à plusieurs égards. Techniquement, il a permis de découvrir et de maîtriser des outils et technologies jusqu'alors inconnus : Docker, PostgreSQL, LittleFS, la conception de PCBs sous EasyEDA, la modélisation 3D sous OnShape, et les réseaux maillés LoRa via Meshtastic. Sur le plan de la conduite de projet, il a imposé une autonomie réelle : analyser un projet sans pouvoir s'appuyer sur ses auteurs initiaux, prendre des décisions d'architecture structurantes, définir des protocoles de tests pour valider la solution, et gérer seul les priorités sur une durée longue sont des compétences qui ne s'acquièrent que par la pratique.

La principale leçon à retenir pour de futurs projets similaires est de définir des protocoles de tests dès le début du développement et de rechercher une confrontation terrain au plus tôt. Construire une architecture sur des hypothèses théoriques sans validation réelle précoce est un risque que ce stage a mis en évidence concrètement.

Les perspectives du projet sont directement liées au déploiement au stade du FCL et à la fiabilisation des communications LoRa. Ce premier test en conditions réelles permettra d'identifier les ajustements nécessaires avant une utilisation lors d'un vrai match, pour lequel le dispositif a été conçu depuis le début.

11 Sources

- Logo UBS
Lien : <https://www-ium.univ-brest.fr/intranet/communication/logos/tutelles-ium/ubs/universite-bretagne-sud-logo.png/view>
- Logo Faculté des Sciences et Sciences de l'ingénieur
Lien : www-facultesciences.univ-ubs.fr
- Logo Lab-STICC
Lien : <https://labsticc.fr/favicon.ico?version=d788779e>
- Photo stade du moustoir
Lien : <https://www.fclorient.bzh/visites-du-moustoir-sessions-completes-pour-demain/>
- Capteur Polar H10
Lien : <https://www.polar.com/en/guide/what-is-good-HRV>
- Capteur MinIMU-9 v6
Lien : <https://botland.store/9dof-imu-sensors/23639-minimu-9-v6-module-with-accelerometer-and-magnetometer-lsm6ds0-and-lis3mdl-pololu-2862.html>
- Capteur INMP441
Lien : <https://www.flyrobo.in/inmp441-mems-high-precision-omnidirectional-microphone-module-i2s-interface-supports-esp32>
- Carte DWM3001CDK
Lien : <https://eu.mouser.com/ProductDetail/Qorvo/DWM3001CDK?qs=QNEbhJQKvYHx2Hn8Cdggjg%3D%3D>
- Carte Xiao seeed studio ESP32 S3 Plus
Lien : <https://www.seeedstudio.com/Seeed-Studio-XIAO-ESP32S3-Plus-p-6361.html>
- Carte Xiao seeed studio EPS32 S3 avec module LoRa Wio-SX1262
Lien : https://wiki.seeedstudio.com/wio_sx1262_with_xiao_esp32s3_kit/
- Schéma de l'architecture du projet initial
Lien : Rapport de projet des étudiants de M2
- Schéma et design des PCBs
Lien : <https://easyeda.com/>
- Design des modèles 3D
Lien : <https://www.onshape.com/fr/>
- Documentation Meshtastic (Limitation légale du duty cycle)
Lien : <https://meshtastic.org/docs/configuration/radio/lora/>

— Dépôt du code source

Lien : <https://github.com/DekkrixX/Data-Monitoring-Football-Fan>

12 Annexes

.1 Structure et organisation du code du projet

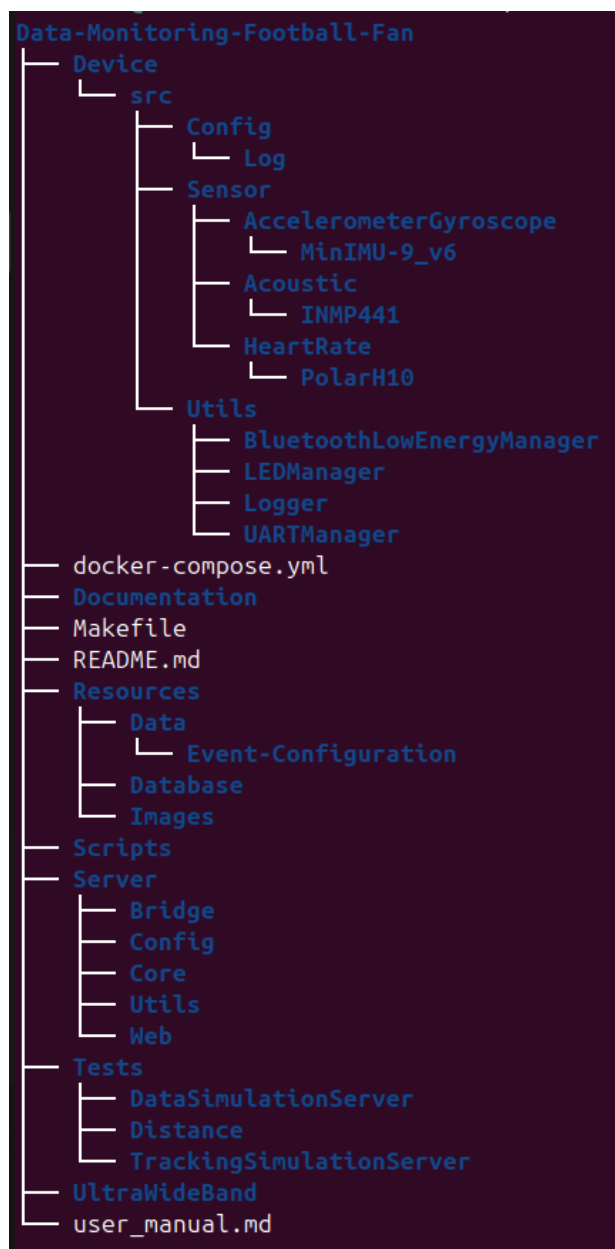


FIGURE 13 – Structure générale du projet

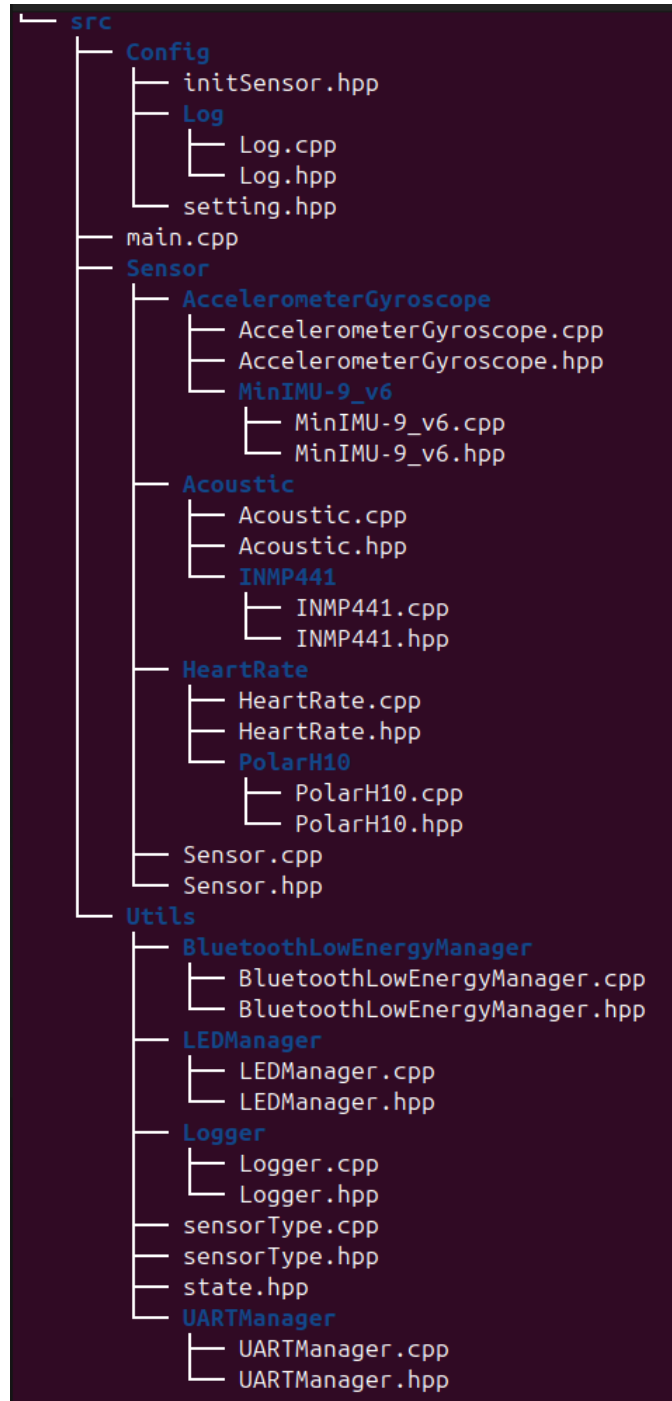


FIGURE 14 – Structure de la partie code firmware des cartes ESP32



FIGURE 15 – Structure de la partie du code du serveur

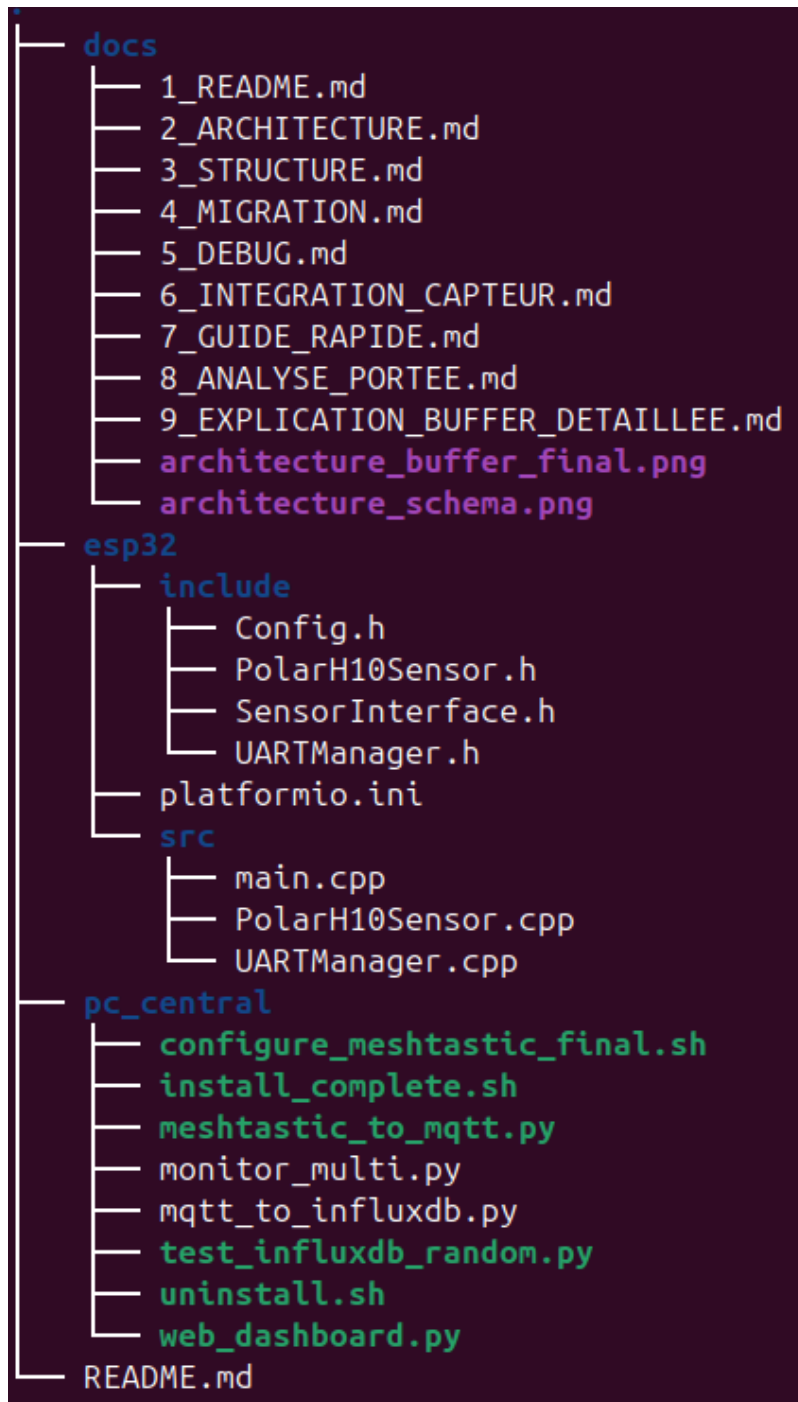


FIGURE 16 – Structure initiale du projet

.2 Capture d'écran de l'interface web du serveur

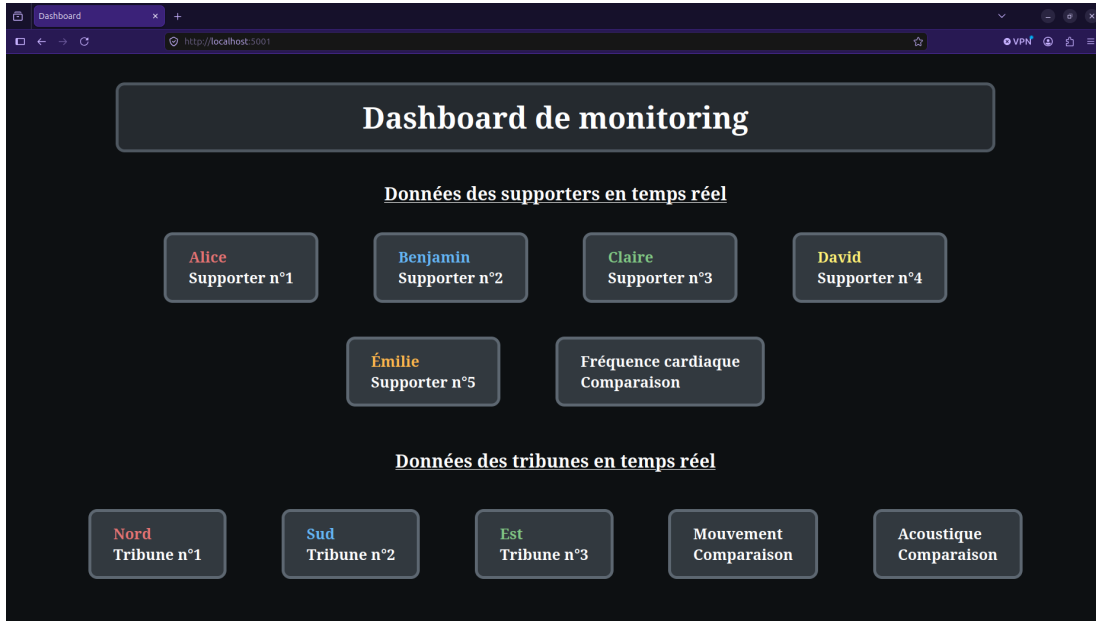


FIGURE 17 – Capture d'écran du dashboard de monitoring des données

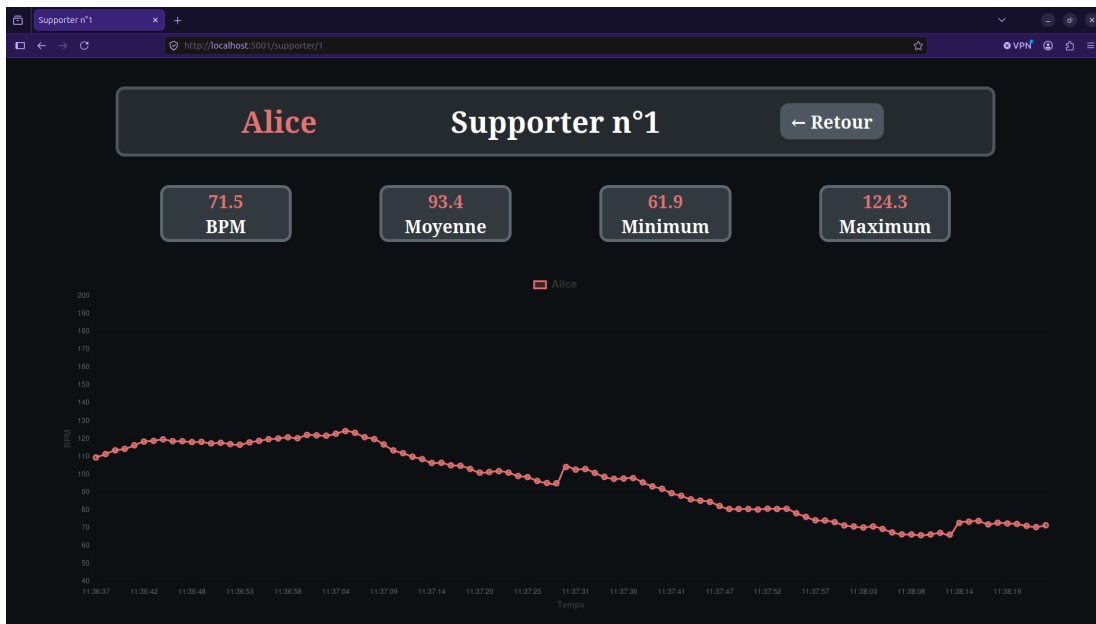


FIGURE 18 – Capture d'écran de la visualisation de la fréquence cardiaque d'un supporter

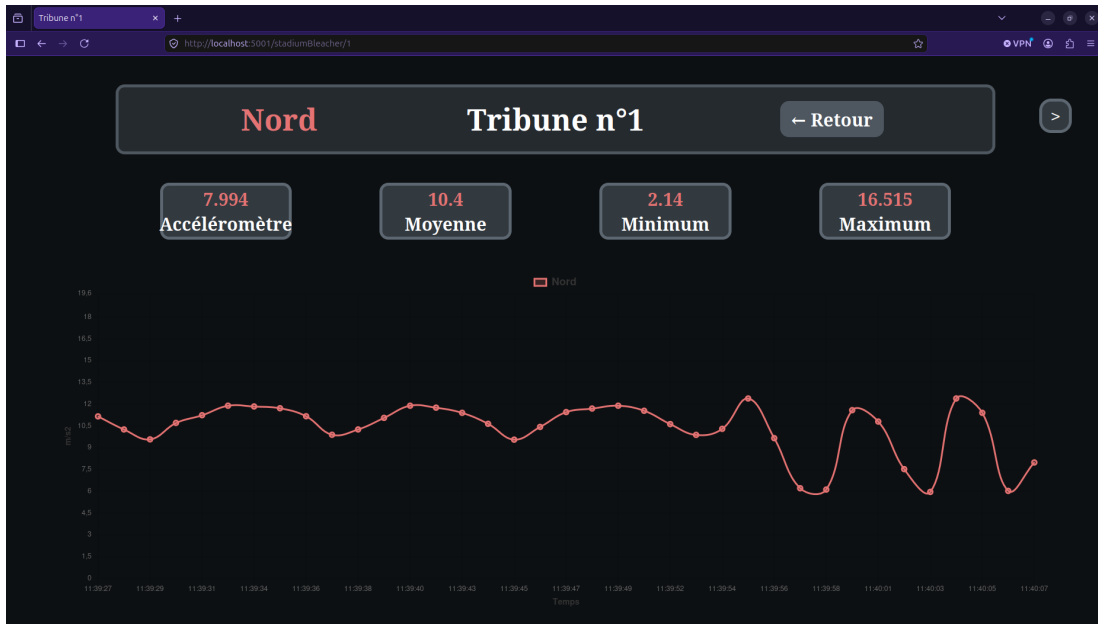


FIGURE 19 – Capture d'écran de la visualisation de l'accélération d'une tribune

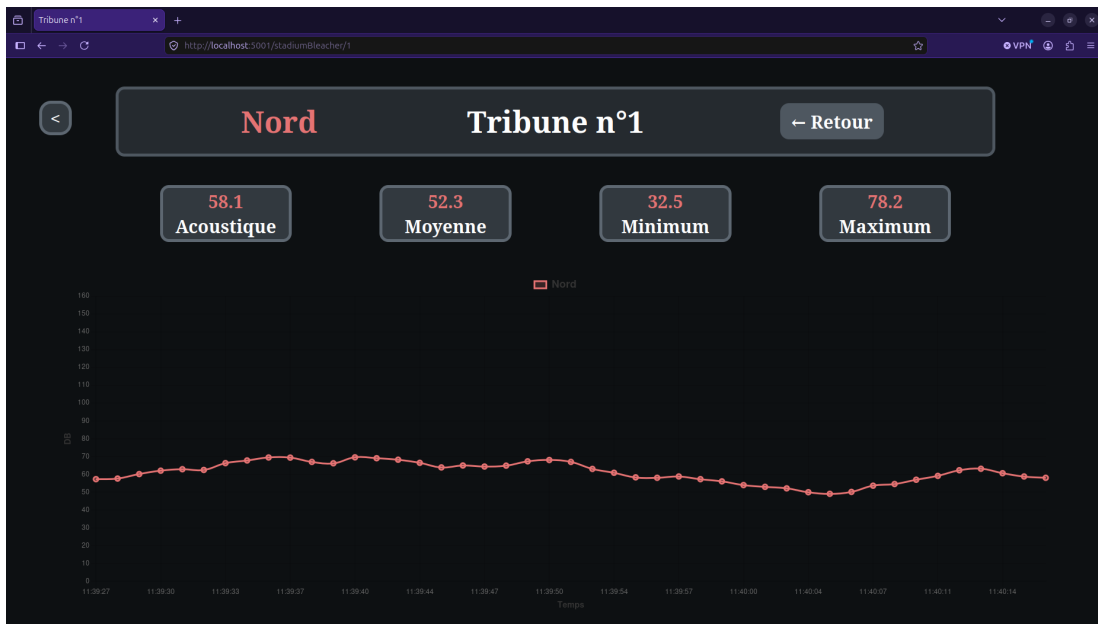


FIGURE 20 – Capture d'écran de la visualisation du niveau acoustique d'une tribune

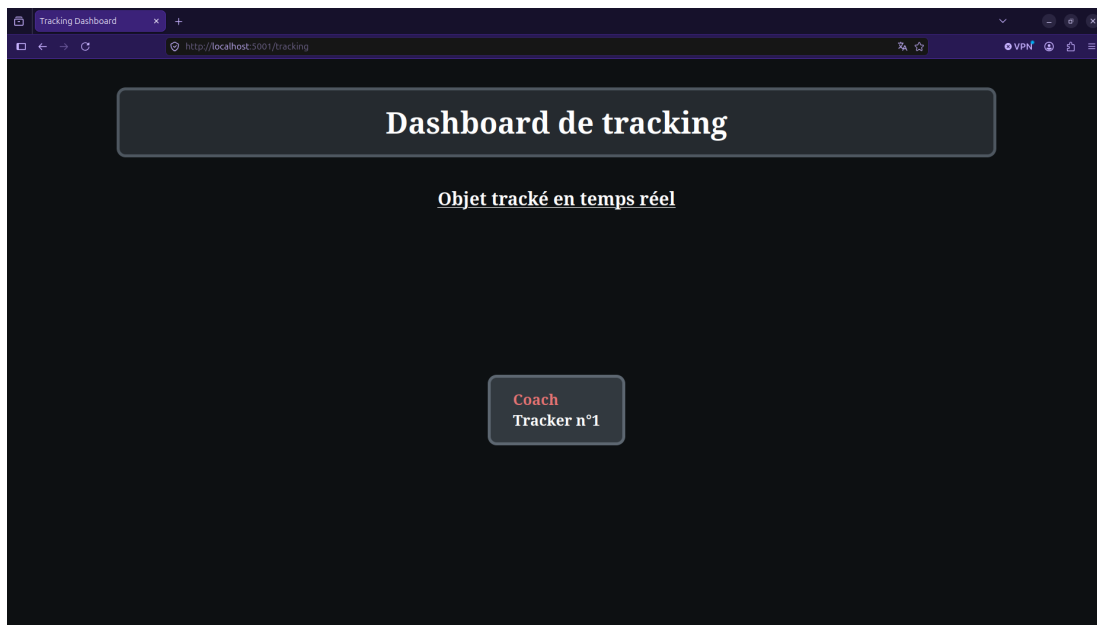


FIGURE 21 – Capture d'écran du dashboard des trackers



FIGURE 22 – Capture d'écran de la visualisation de la position d'un tracker

The schematic diagram illustrates the Data-Monitoring-Football-Fan module, which consists of two identical circuit blocks connected to a common VCC and GND. Each block contains a microcontroller (Xiao ESP32S3-PLUS) and various peripheral components like resistors, capacitors, and LEDs. The components are labeled with their values and pin numbers.

Block 1 Components:

- VCC
- GND
- U1: XIAO-ESP32S3-PLUS
- U1A: XIAO-ESP32S3-PLUS
- U1B: XIAO-ESP32S3-PLUS
- U1C: XIAO-ESP32S3-PLUS
- U1D: XIAO-ESP32S3-PLUS
- U1E: XIAO-ESP32S3-PLUS
- U1F: XIAO-ESP32S3-PLUS
- U1G: XIAO-ESP32S3-PLUS
- U1H: XIAO-ESP32S3-PLUS
- U1I: XIAO-ESP32S3-PLUS
- U1J: XIAO-ESP32S3-PLUS
- U1K: XIAO-ESP32S3-PLUS
- U1L: XIAO-ESP32S3-PLUS
- U1M: XIAO-ESP32S3-PLUS
- U1N: XIAO-ESP32S3-PLUS
- U1O: XIAO-ESP32S3-PLUS
- U1P: XIAO-ESP32S3-PLUS
- U1Q: XIAO-ESP32S3-PLUS
- U1R: XIAO-ESP32S3-PLUS
- U1S: XIAO-ESP32S3-PLUS
- U1T: XIAO-ESP32S3-PLUS
- U1U: XIAO-ESP32S3-PLUS
- U1V: XIAO-ESP32S3-PLUS
- U1W: XIAO-ESP32S3-PLUS
- U1X: XIAO-ESP32S3-PLUS
- U1Y: XIAO-ESP32S3-PLUS
- U1Z: XIAO-ESP32S3-PLUS

Block 2 Components:

- VCC
- GND
- U2: XIAO-ESP32S3-PLUS
- U2A: XIAO-ESP32S3-PLUS
- U2B: XIAO-ESP32S3-PLUS
- U2C: XIAO-ESP32S3-PLUS
- U2D: XIAO-ESP32S3-PLUS
- U2E: XIAO-ESP32S3-PLUS
- U2F: XIAO-ESP32S3-PLUS
- U2G: XIAO-ESP32S3-PLUS
- U2H: XIAO-ESP32S3-PLUS
- U2I: XIAO-ESP32S3-PLUS
- U2J: XIAO-ESP32S3-PLUS
- U2K: XIAO-ESP32S3-PLUS
- U2L: XIAO-ESP32S3-PLUS
- U2M: XIAO-ESP32S3-PLUS
- U2N: XIAO-ESP32S3-PLUS
- U2O: XIAO-ESP32S3-PLUS
- U2P: XIAO-ESP32S3-PLUS
- U2Q: XIAO-ESP32S3-PLUS
- U2R: XIAO-ESP32S3-PLUS
- U2S: XIAO-ESP32S3-PLUS
- U2T: XIAO-ESP32S3-PLUS
- U2U: XIAO-ESP32S3-PLUS
- U2V: XIAO-ESP32S3-PLUS
- U2W: XIAO-ESP32S3-PLUS
- U2X: XIAO-ESP32S3-PLUS
- U2Y: XIAO-ESP32S3-PLUS
- U2Z: XIAO-ESP32S3-PLUS

FIGURE 23 – Schéma du PCB du noeud LoRa

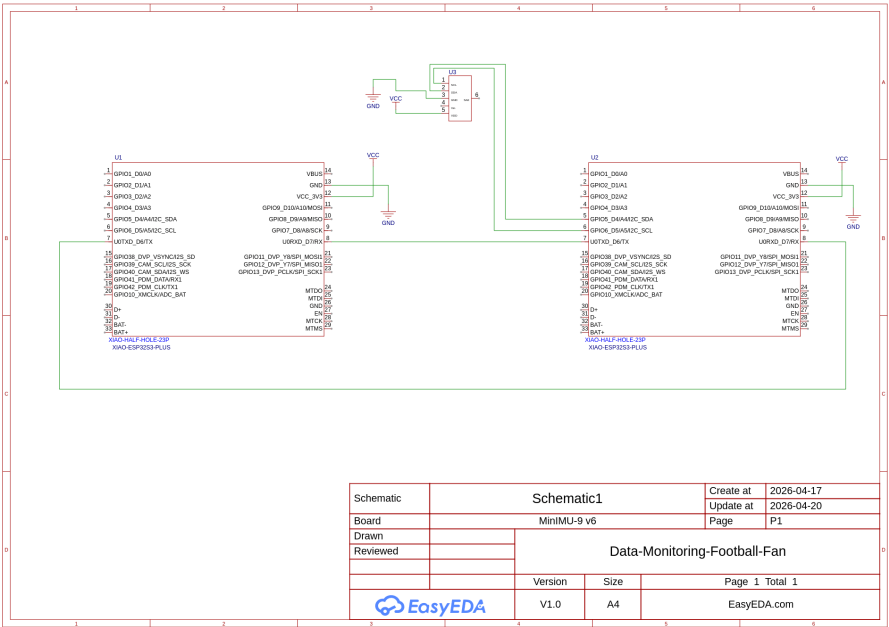


FIGURE 24 – Schéma du PCB du noeud LoRa avec le capteur MinIMU-9 v6

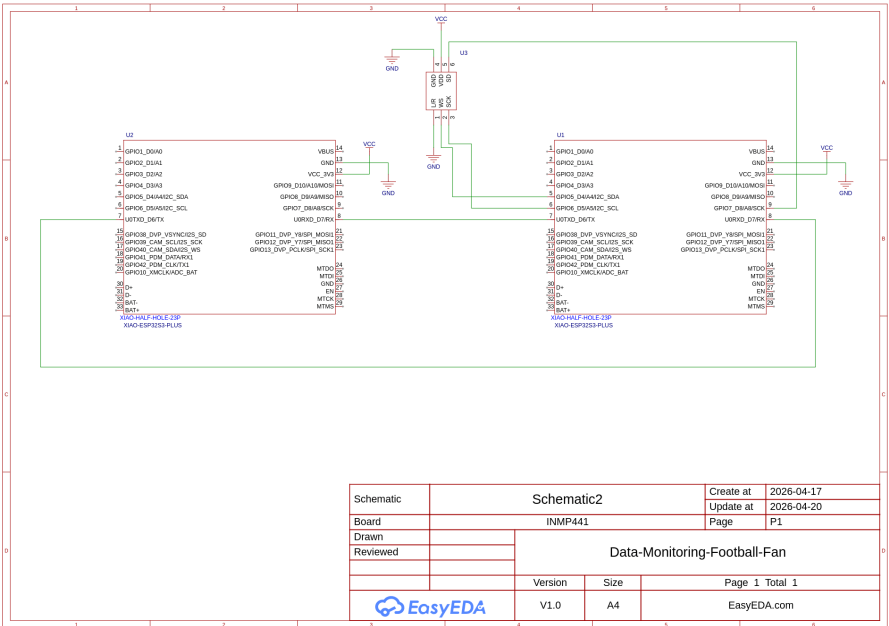


FIGURE 25 – Schéma du PCB du noeud LoRa avec le capteur INMP441

.4 Design des PCBs

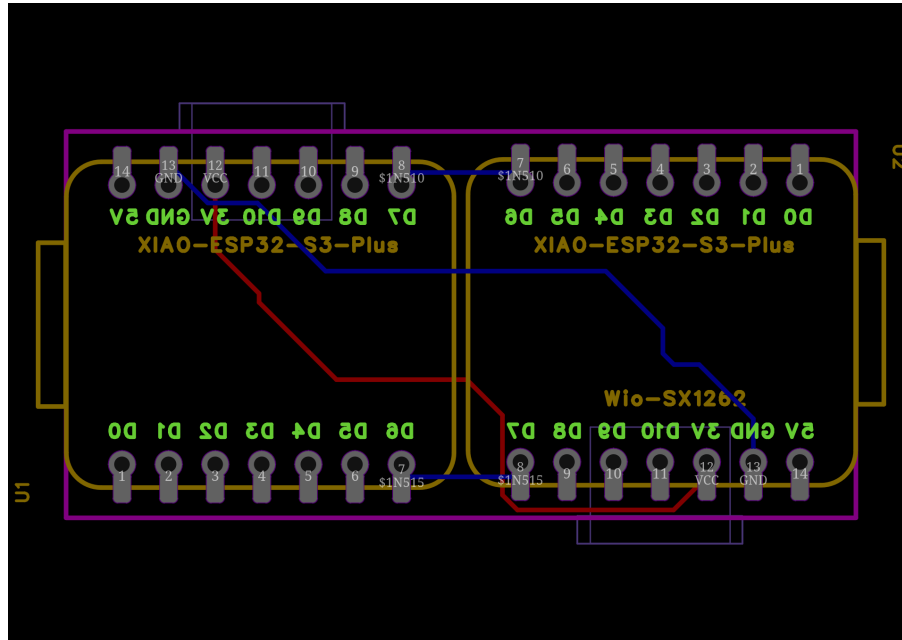


FIGURE 26 – Design du PCB du noeud LoRa

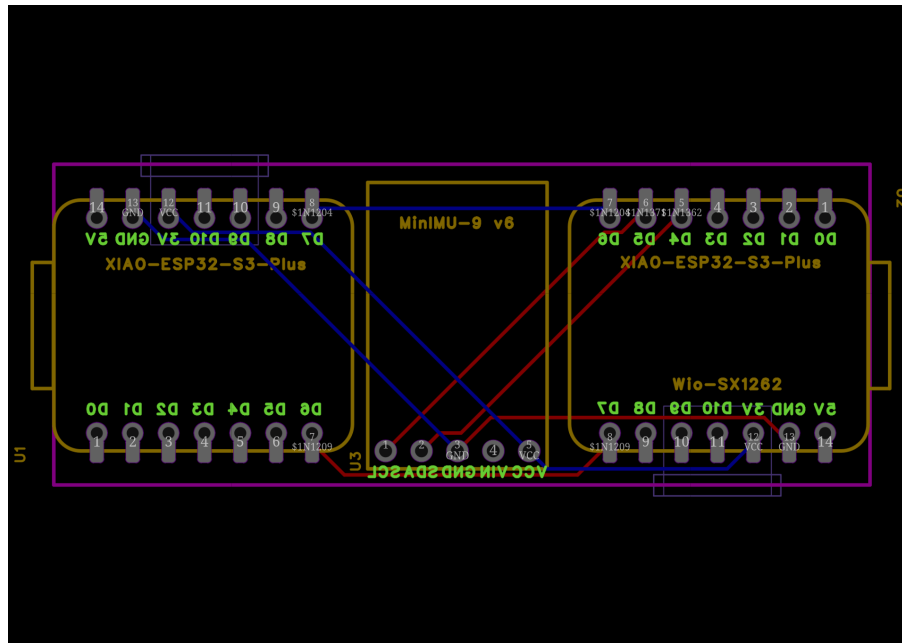


FIGURE 27 – Design du PCB du noeud LoRa avec le capteur MinIMU-9 v6

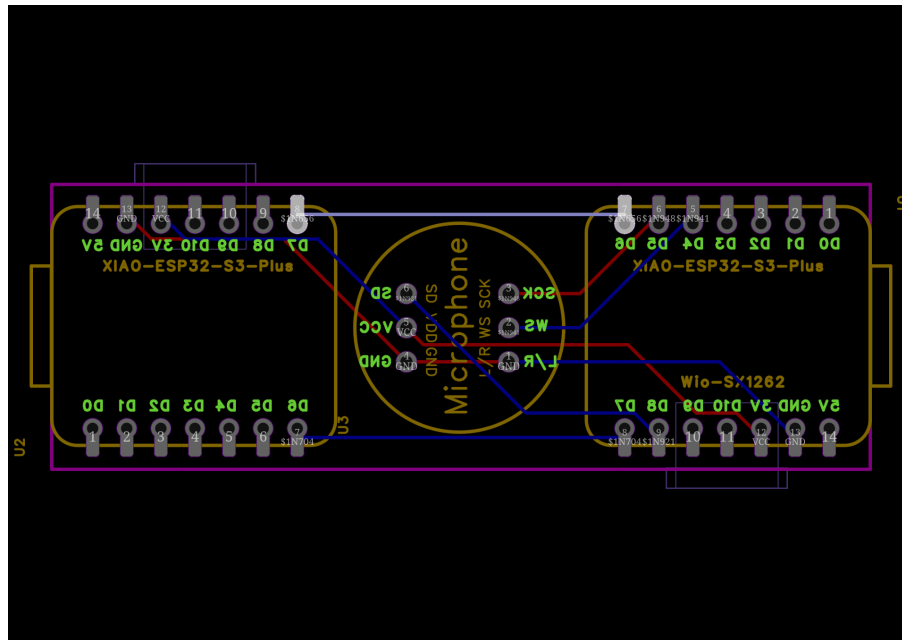


FIGURE 28 – Design du PCB du noeud LoRa avec le capteur INMP441

.5 Design des modèles 3D

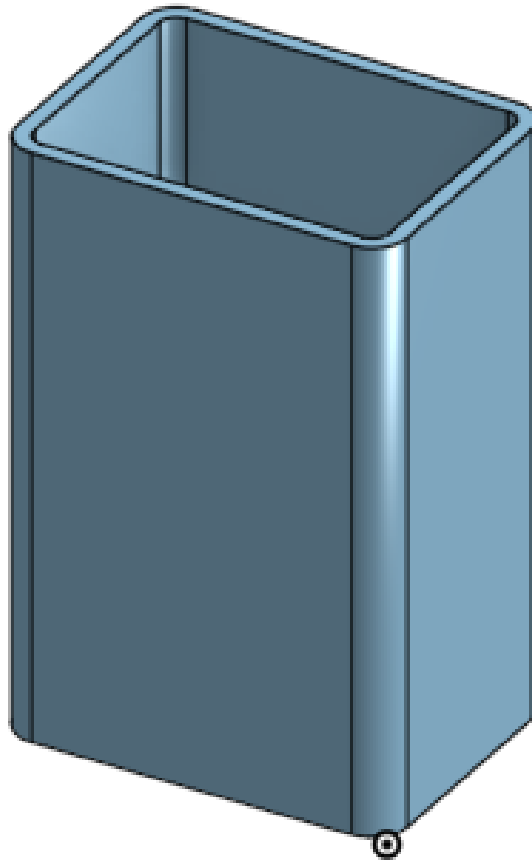


FIGURE 29 – Design du modèle 3D du boîtier

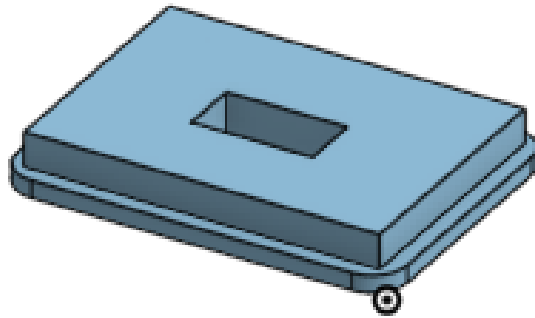


FIGURE 30 – Design du modèle 3D du couvercle

.6 Document de tests

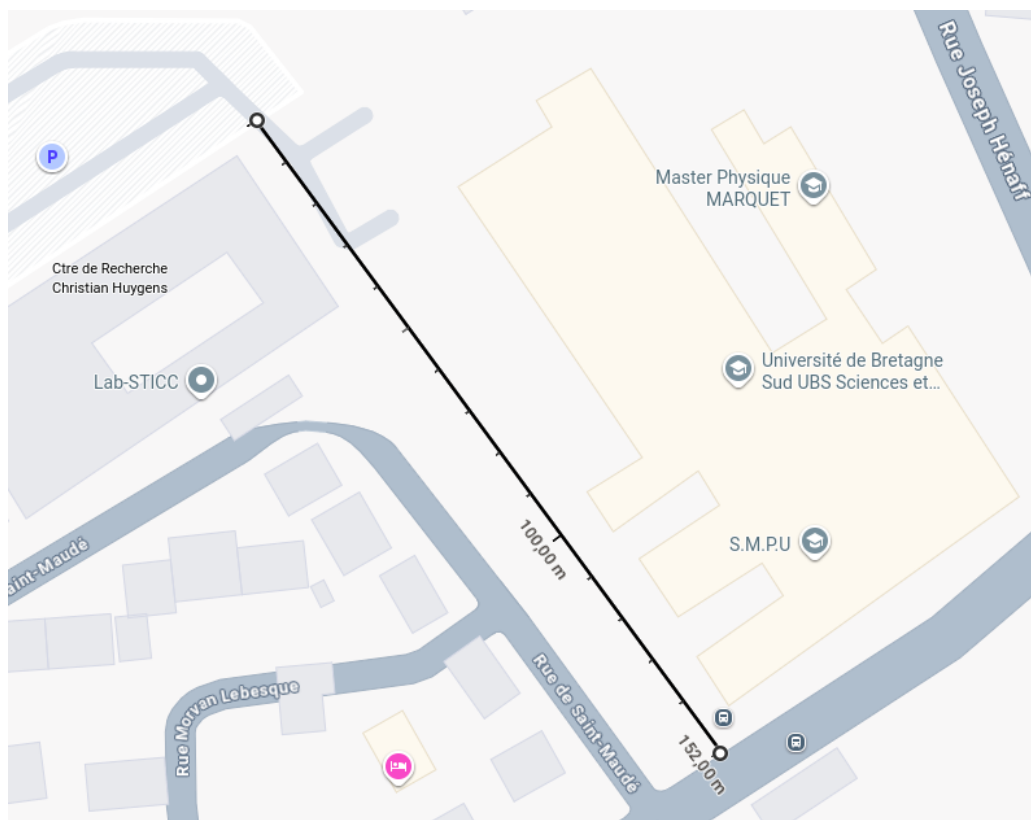


FIGURE 31 – Mesure de la distance de test de portée LoRa sur le parking du centre de recherche et de science 1



FIGURE 32 – Mesure de la distance de test de portée LoRa sur le port de Lorient

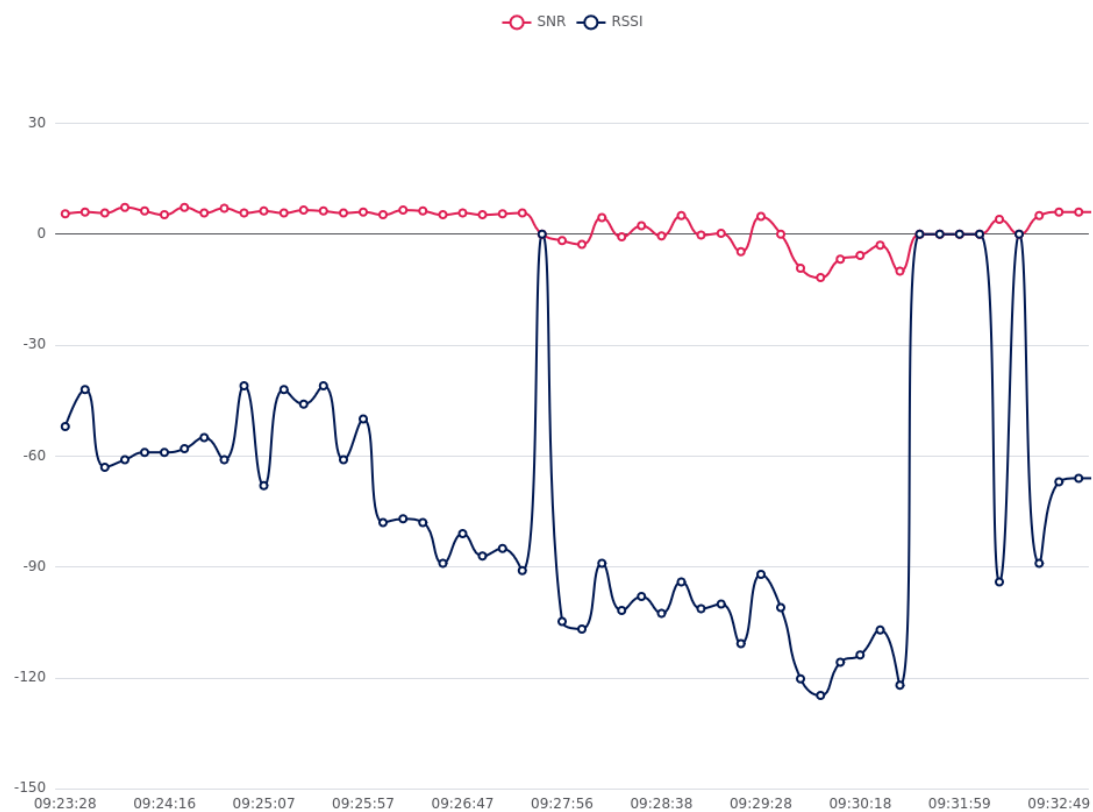


FIGURE 33 – Chemin en ligne de vue avec distance maximum de 250m

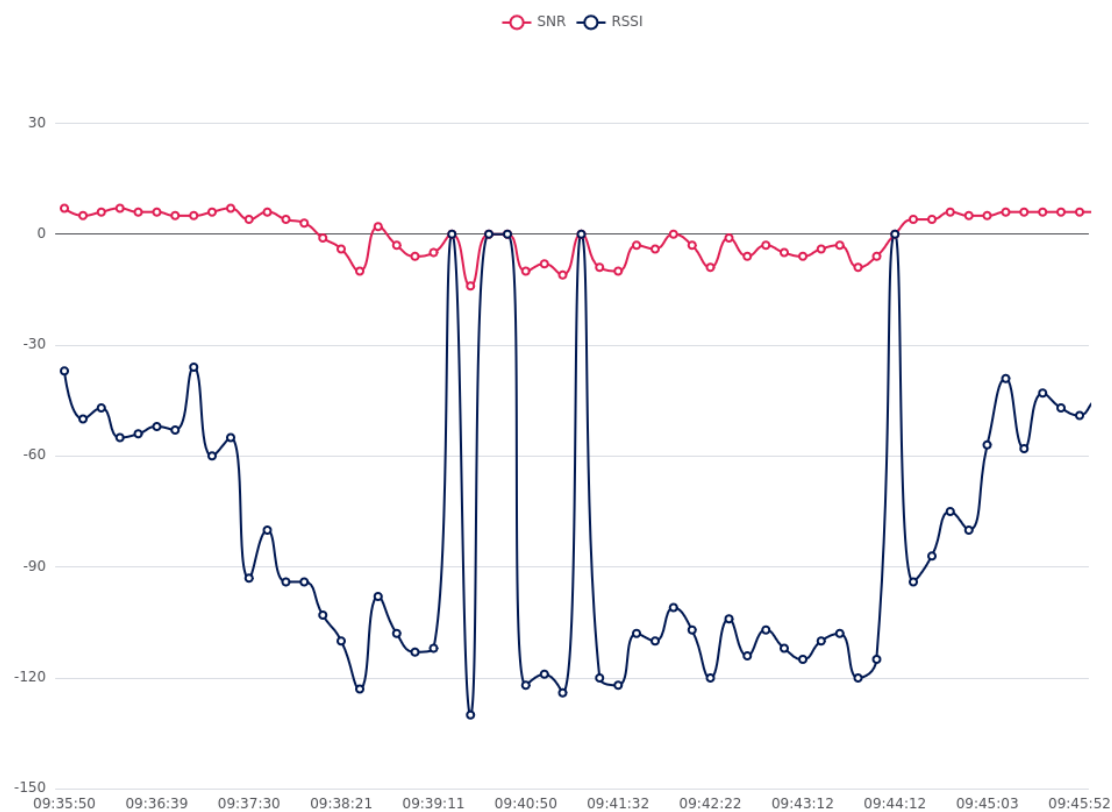


FIGURE 34 – Chemin en ligne de vue avec distance maximum de 250m

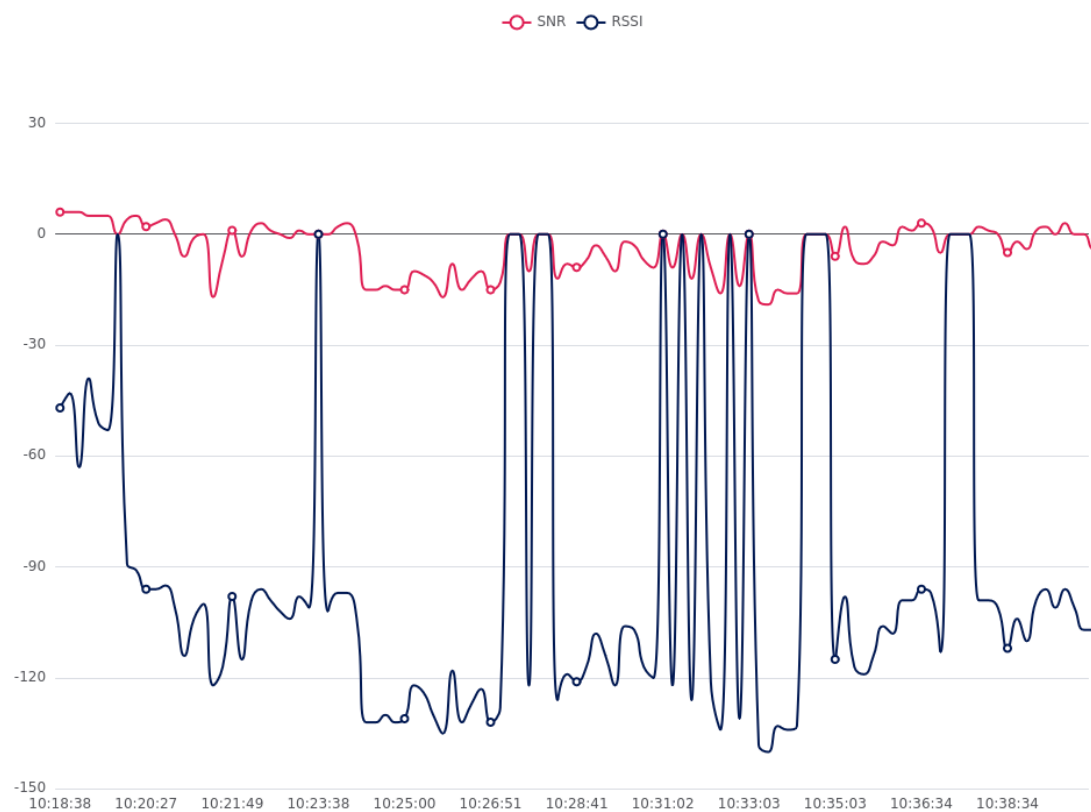


FIGURE 35 – Chemin hors ligne de vue avec distance maximum de 800m



FIGURE 36 – Chemin hors ligne de vue avec distance maximum de 800m et puissance d'émission doublée